

FAST-FORWARD FALCON (SPEED)

- Dynamic symbol of offensive speed & evasion
- Combines aggressive systems performance with agile networking

n for Durin

finitive guide to the Zen programming lan

Zen Language Project

Zen 3.0 · ISBN 978-0-00000-000-0

Contents

1 Zen for Dummies: Complete Edition **5**
1.1 Table of Contents . . . 5
1.2 About This Book . . . 7

2 Chapter 2: Data Types **8**
2.1 2.1 The type system at a glance . . . 8
2.2 2.2 Literals . . . 9
2.3 2.3 Type coercion matrix . . . 12
2.4 2.4 Truthiness . . . 13
2.5 2.5 Equality vs identity . . . 13
2.6 2.6 Numeric edge cases . . . 14
2.7 2.7 Type conversion reference . . . 14
2.8 2.8 Common pitfalls . . . 15
2.9 2.9 Quick reference . . . 15
2.10 2.10 Exercises . . . 16

3 Chapter 3: Operators and Expressions **17**
3.1 3.1 Operator precedence (high to low) . . . 17
3.2 3.2 Arithmetic . . . 17
3.3 3.3 String concatenation and repetition . . . 18
3.4 3.4 Comparison . . . 18
3.5 3.5 Logic . . . 19
3.6 3.6 Safe member access: `?.` (optional chaining) . . . 19
3.7 3.7 Nullish coalescing: `??` . . . 20
3.8 3.8 Conditional expression (ternary) . . . 20
3.9 3.9 Assignment and augmented assignment . . . 20
3.10 3.10 `is` — type membership . . . 21
3.11 3.11 `in` / `!in` — membership . . . 21
3.12 3.12 Quick reference . . . 21
3.13 3.13 Exercises . . . 22

4 Chapter 5: Functions **23**
4.1 5.1 Declaration and calling . . . 23
4.2 5.2 Return statements . . . 23
4.3 5.3 Default arguments . . . 24
4.4 5.4 Variadic behavior . . . 24
4.5 5.5 Functions as first-class values . . . 25
4.6 5.6 Closures . . . 25
4.7 5.7 Higher-order functions . . . 26
4.8 5.8 Anonymous functions (lambdas) . . . 26
4.9 5.9 `return` in closures . . . 27
4.10 5.10 Common pitfalls . . . 27
4.11 5.11 Quick reference . . . 27
4.12 5.12 Exercises . . . 28

5 Chapter 6: Lambdas and Closures **29**

5.1 6.1 Lambda expressions . . . 29
5.2 6.2 Closure capture semantics . . . 29
5.3 6.3 The loop-variable capture trap . . . 30
5.4 6.4 `return` target in closures . . . 31
5.5 6.5 Closures and reference counting . . . 31
5.6 6.6 Closures in collections . . . 32
5.7 6.7 Closures and threads . . . 32
5.8 6.8 JIT lowering note . . . 32
5.9 6.9 Quick reference . . . 33
5.10 6.10 Exercises . . . 33

6 Chapter 11: Error Handling 34
6.1 11.1 Catching errors: `try` / `catch` . . . 34
6.2 11.2 Reading the real failure: `catch as e` . . . 34
6.3 11.3 Error fields: `e.msg`, `e.code`, `e.val` . . . 35
6.4 11.4 String + error concatenation . . . 35
6.5 11.5 Catching specific error types . . . 35
6.6 11.6 Raising errors: `throw` . . . 36
6.7 11.7 Cleanup: `finally` . . . 37
6.8 11.8 `exit()` and `finally` . . . 37
6.9 11.9 Nested handlers . . . 38
6.10 11.10 Errors from callees . . . 38
6.11 11.11 Error types . . . 38
6.12 11.12 Full error-code reference . . . 39
6.13 11.13 Common pitfalls . . . 39
6.14 11.14 Quick reference . . . 39
6.15 11.15 Exercises . . . 40

7 Chapter 25: crypto Module — Complete Guide 41
7.1 25.1 Symmetric encryption . . . 41
7.2 25.2 Asymmetric encryption (RSA) . . . 42
7.3 25.3 Digital signatures . . . 43
7.4 25.4 Key agreement (X25519) . . . 44
7.5 25.5 Hashing and HMAC . . . 45
7.6 25.6 Key derivation (PBKDF2) . . . 46
7.7 25.7 JWT (JSON Web Tokens) . . . 46
7.8 25.8 OTP (TOTP/HOTP) . . . 47
7.9 25.9 X.509 certificates . . . 48
7.10 25.10 Constant-time compare and secure zero . . . 49
7.11 25.11 Algorithm policy (FIPS-ish mode) . . . 49
7.12 25.12 Quick reference . . . 50
7.13 25.13 Exercises . . . 51

8 Chapter 26: Networking Fundamentals 52
8.1 26.1 Networking module map . . . 52
8.2 26.2 IP addressing . . . 52
8.3 26.3 Ports . . . 53
8.4 26.4 TCP vs UDP . . . 54

8.5 26.5 The OSI model and Zen . . . 54
8.6 26.6 Sockets: file descriptors . . . 55
8.7 26.6 Address families . . . 55
8.8 26.6 Timeouts . . . 56
8.9 26.6 Byte order . . . 56
8.10 26.7 Quick reference . . . 56
8.11 26.8 Exercises . . . 57

9 Chapter 27: socket Module **58**
9.1 27.1 Socket creation . . . 58
9.2 27.2 Socket handle operations . . . 58
9.3 27.3 TCP client . . . 59
9.4 27.4 TCP server . . . 60
9.5 27.5 UDP . . . 61
9.6 27.6 Socket options . . . 62
9.7 27.7 Common patterns . . . 62
9.8 27.8 Common pitfalls . . . 64
9.9 27.8 Quick reference . . . 64
9.10 26.9 Exercises . . . 65

10 Chapter 28: HTTP Protocol **66**
10.1 28.1 HTTP basics . . . 66
10.2 28.2 The `http` module . . . 66
10.3 28.3 Making requests with headers . . . 67
10.4 28.4 POST with body . . . 68
10.5 28.5 Status codes . . . 68
10.6 28.6 HTTP vs HTTPS . . . 68
10.7 28.7 Common patterns . . . 69
10.8 28.8 Common pitfalls . . . 70
10.9 28.9 Quick reference . . . 70
10.10 28.10 Exercises . . . 70

11 Chapter 31: HTTPS & SSL/TLS Certificates **71**
11.1 31.1 The `tls` module . . . 71
11.2 31.2 Certificate pinning . . . 71
11.3 31.3 Plain TLS vs pinned TLS . . . 72
11.4 31.4 Complete HTTPS client example . . . 72
11.5 31.5 Error handling . . . 74
11.6 31.6 Quick reference . . . 74
11.7 31.7 Exercises . . . 74

12 Chapter 35: Threading Fundamentals **76**
12.1 35.1 The `thread` module . . . 76
12.2 35.2 Thread lifecycle . . . 77
12.3 35.3 Thread-local data . . . 77
12.4 35.3 Multiple threads . . . 78
12.5 35.4 Common pitfalls . . . 78
12.6 35.5 Quick reference . . . 78

12.7 35.6 Exercises 78

13 Chapter 36: Channels **80**
13.1 36.1 The `channel` module 80
13.2 36.2 Blocking semantics 80
13.3 36.3 Producer-consumer 81
13.4 36.4 Multiple producers and consumers 82
13.5 36.5 Channels and shared values 83
13.6 36.6 Common pitfalls 83
13.7 36.7 Quick reference 83
13.8 36.8 Exercises 84

14 Chapter 42: pymport - Python Interoperability **85**
14.1 42.1 Why `pymport` (and not just `import`)? 85
14.2 42.2 Syntax and Binding 85
14.3 42.3 Building Zen with Python Support 86
14.4 42.4 Environment Setup (venv + PYTHONPATH) 86
14.5 42.5 Dependency Sync with `zen py-sync` and `requirements.txt` 86
14.6 42.6 Using Python from Zen 87
14.7 42.7 Error handling 89
14.8 42.8 Limits and Non-Goals (v1) 90
14.9 42.9 Working Examples (tested) 90
14.10 42.10 Troubleshooting 90
14.11 42.11 References 91
14.12 42.12 Quick reference 91
14.13 42.13 Exercises 92

15 Chapter 44: Web Scraping & Automation **93**
15.1 44.1 Setup 93
15.2 44.2 Run the whole suite 93
15.3 44.3 1) Scrape + Download with `requests` + `bs4` (pymport) 94
15.4 44.4 2) Stateful Scraping + Download with `mechanicalsoup` (pymport) 96
15.5 44.5 3) Browser Automation with Selenium (pymport) 97
15.6 44.6 4) Low-Level Control: CDP over raw TCP + WebSocket sockets 100
15.7 44.7 Tips and Gotchas 104
15.8 44.8 Verified Artifacts 105
15.9 44.9 Appendix: Tested Scripts 105

16 Chapter 45: REST API Client **106**
16.1 45.1 Basic GET request 106
16.2 45.2 POST with JSON body 106
16.3 45.3 Reusable API client 106
16.4 45.4 Authentication 108
16.5 45.5 Pagination 108
16.6 45.6 Rate limiting 109
16.7 45.7 Error handling 110
16.8 45.8 Complete CLI example 111
16.9 45.9 Common pitfalls 111

16.1045.10 Quick reference . . . 111
16.1145.11 Exercises . . . 112

17 Chapter 46: File Processing Tools 113
17.1446.1 Reading text files . . . 113
17.2446.2 Writing text files . . . 113
17.3446.3 Binary files . . . 113
17.4446.4 CSV processing (manual) . . . 114
17.5446.5 JSON files . . . 114
17.6446.6 Line processing pipeline . . . 115
17.7446.7 Temporary files . . . 115
17.8446.8 Directory walking . . . 115
17.9446.9 File info . . . 116
17.1046.10 Quick reference . . . 116
17.1146.11 Exercises . . . 117

18 Chapter 47: Network Security Tools 118
18.1447.1 Port Scanner . . . 118
18.244. Subdomain Finder . . . 118
18.344. HTTP Header Checker . . . 119
18.4445 Port Scan Report . . . 120
18.5446 HTTP Header Analysis . . . 120
18.6447 DNS Reconnaissance . . . 121
18.7448 Common pitfalls . . . 122
18.8449 Quick reference . . . 122
18.94410 Exercises . . . 122

19 Chapter 48: Capstone Projects 124
19.1481 Capstone 1: Web Metadata Archiver . . . 124
19.2482 Capstone 2: Password Strength Checker . . . 125
19.3483 Capstone 3: Log Analyzer . . . 127
19.4484 Capstone 4: Site Uptime Monitor . . . 129
19.5485 Running the Capstones . . . 131
19.6486 Quick reference . . . 131
19.7487 Exercises . . . 131

1 Zen for Dummies: Complete Edition

The Definitive Guide to the Zen Programming Language

1.1 Table of Contents

This comprehensive guide is organized into detailed sections covering every aspect of Zen.

1.1.1 Part 1: Fundamentals

1. Getting Started - Installation, first program, REPL

2. Data Types - Primitives, collections, type system
3. Operators and Expressions - Arithmetic, logical, bitwise
4. Control Flow - Conditionals, loops, branching
5. Functions - Defining, calling, parameters, returns
6. Lambdas and Closures - Anonymous functions, higher-order functions

1.1.2 Part 2: Core Concepts

7. Strings Deep Dive - Manipulation, formatting, methods
8. Lists and Arrays - Operations, methods, iteration
9. Dictionaries and Maps - Key-value pairs, methods
10. Modules and Imports - Creating, importing, module system
11. Error Handling - Try/catch, throw, debugging

1.1.3 Part 3: Object-Oriented & Advanced Programming

12. Classes and Objects - OOP, inheritance, polymorphism
13. File I/O - Reading, writing, `with` statement
14. Memory and References - Values, references, ownership

1.1.4 Part 4: Standard Library Modules (Detailed)

15. math Module - Mathematical functions and constants
16. random Module - Random numbers and selection
17. string Module - String utilities
18. collections Module - Advanced data structures
19. json Module - JSON parsing and serialization
20. datetime/time Module - Time and date operations
21. pathlib & os Module - File system operations
22. re (Regex) Module - Pattern matching
23. base64 & binascii - Encoding/decoding
24. hashlib Module - Hashing algorithms
25. crypto Module - Complete Guide - Cryptography, encryption, JWT, X.509

1.1.5 Part 5: Networking & Protocols

26. Networking Fundamentals - Sockets, TCP/IP basics
27. socket Module - Low-level socket programming
28. HTTP Protocol - HTTP/1.1, requests, headers
29. requests Module - High-level HTTP client
30. http Server Module - Building HTTP servers
31. HTTPS & SSL/TLS Certificates - Secure connections, certificates
32. DNS Module - Domain name resolution
33. url Module - URL parsing and manipulation
34. Raw Sockets - Packet-level networking

1.1.6 Part 6: Concurrency & Threading

35. Threading Fundamentals - Threads, spawning, execution
36. Channels - Inter-thread communication
37. Mutexes & Locks - Synchronization primitives
38. Thread Safety & Patterns - Best practices

1.1.7 Part 7: FFI, System & Interoperability

39. FFI Fundamentals - Foreign Function Interface basics
40. FFI in Practice - Calling C functions, handles
41. sys Module - System information and operations
42. pymport - Python Interoperability - Using Python from Zen (setup, sync, examples)

1.1.8 Part 8: Practical Projects

43. CLI Tools - Building command-line applications
44. Web Scraping & Automation - Bots, scrapers, automation
45. REST API Client - API integration projects
46. File Processing Tools - Data processing pipelines
47. Network Security Tools - Ethical hacking utilities
48. Capstone Projects - Full applications

1.1.9 Appendices

- Appendix A: Built-in Functions
- Appendix B: Error Codes Reference
- Appendix C: Zen CLI Reference
- Appendix D: Standard Library Index
- Appendix E: Quick Reference Card

1.2 About This Book

This book is written for complete beginners but goes deep into every aspect of Zen. Each module gets its own detailed chapter with theory, examples, common pitfalls, and exercises.

2 Chapter 2: Data Types

Zen is a dynamically typed, strongly typed language. Every value carries a type tag at runtime, operations check operand types, and implicit conversions are minimal and explicit. This chapter is the definitive reference for every literal form, every type, every coercion, and every edge case in Zen's value system.

2.1 2.1 The type system at a glance

Zen values are 64-bit NaN-boxed tagged unions. There are **16 language types**:

#	Type	Kind	Storage	`typeof()` returns
1	`nil`	primitive	inline	`"nil"`
2	`bool`	primitive	inline	`"bool"`
3	`int`	primitive	inline (i64)	`"int"`
4	`float`	primitive	inline (f64)	`"float"`
5	`string`	heap	`Arc<str>`	`"string"`
6	`bytes`	heap	`Arc<[u8]>`	`"bytes"`
7	`list`	heap	`Vec<Value>`	`"list"`
8	`dict`	heap	`IndexMap`	`"dict"`
9	`set`	heap	`IndexSet`	`"set"`
10	`tuple`	heap	`Vec<Value>`	`"tuple"`
11	`func`	heap	`FuncObj`	`"func"`
12	`class`	heap	`ClassObj`	`"class"`
13	`instance`	heap	`InstObj`	`"instance"`
14	`cell`	heap	`RefCell<Value>`	`"cell"`
15	`error`	heap	`ErrObj`	`"error"`
16	`foreign`	heap	`ForeignObj`	`"foreign"`
17	`channel`	heap	`ChanObj`	`"channel"`
18	`mutex`	heap	`MutexObj`	`"mutex"`
19	`handle`	heap	`HandleObj`	`"handle"`

> Note: internally there are 16 machine tags over a NaN-boxed `u64`; small primitives (`nil`, `bool`, `int`, `float`) are stored inline in the payload bits. Larger values (strings, lists, dicts, etc.) are heap objects with manual reference counting (`retain`/`release`).

2.1.1 Type introspection

```
typeof(42)        # "int"
typeof(3.14)      # "float"
typeof("hello")   # "string"
typeof(true)      # "bool"
typeof(nil)       # "nil"
typeof([1, 2])    # "list"
typeof({"a": 1})  # "dict"
typeof(fn() {})   # "func"
```

`is` checks type membership:

```
42 is int         # true
"hello" is string # true
nil is nil        # true
42 is string      # false
```

`is` compares against a type *name* (a string literal or value), not against another value's type:

```
let x = 42
x is typeof(x)    # true — compares "int" is "int"
```

2.2 2.2 Literals

2.2.1 Nil and booleans

```
let nothing = nil
let yes = true
let no = false
```

`nil` is the absence of a value. It is falsy. `true`/`false` are the only boolean values.

2.2.2 Integers

Integer literals are 64-bit signed (`i64`). Decimal, hex (`0x`), octal (`0o`), and binary (`0b`) bases are supported. Underscores may be used as digit separators.

```
let dec = 42
let hex = 0x2A          # 42
let oct = 0o52          # 42
let bin = 0b101010      # 42
let big = 1_000_000_000 # 1000000000
let neg = -17
```

Integer range: `-9,223,372,036,854,775,808` to `9,223,372,036,854,775,807` (`i64::MIN` to `i64::MAX`). Overflow in checked contexts panics; in release builds overflow wraps (two's complement).

```
let max = 9223372036854775807   # i64::MAX
let min = -9223372036854775808  # i64::MIN
```

2.2.3 Floats

Float literals are 64-bit IEEE-754 (`f64`). A decimal point or exponent is required.

```
let pi = 3.14159
let big = 1.5e10      # 15000000000.0
```

```
let small = 2.5e-3    # 0.0025
let hex_f = 0x1.8p1   # 3.0 (hex float)
```

Special values:

```
let inf = 1e400       # overflow → infinity
let neg_inf = -1e400  # −infinity
let nan = 0.0 * inf   # not-a-number
```

NaN semantics: - `nan != nan` is `true` (IEEE-754) - `nan < anything` is `false`; `nan > anything` is `false` - `typeof(nan)` → `"float"` - Use `x != x` to test for NaN: `let is_nan = (x != x)`

```
let bad = 0.0 / 0.0   # NaN
print(bad == bad)     # false
print(bad != bad)     # true  ← NaN detection idiom
print(inf > 1e308)    # true
print(-inf < -1e308)  # true
```

2.2.4 Strings

String literals use double or single quotes. Escape sequences: `\n`, `\t`, `\\`, `\'`, `\"`, `\0`, `\xHH` (hex byte), `\u{HHHH}` (Unicode scalar).

```
let s1 = "hello"
let s2 = 'world'
let s3 = "it's a nice day"
let s4 = "say \"hello\""
let s5 = "line1\nline2\tline3"
let s6 = "byte: \x41"    # "byte: A"
let s7 = "emoji: \u{1F600}"  # "emoji: □"
```

Triple-quoted strings (multi-line, raw):

```
let poem = """Zen is fast,
native, and safe.
No escaping needed here."""
```

Strings are immutable (`Arc<str>`). Indexing `s[i]` returns the character at byte offset `i` as a one-character string; slicing `s[a:b]` returns a substring. Indexing out of bounds raises `IndexError`.

```
let s = "Zen"
print(s[0])   # "Z"
print(s[-1])  # "n"
print(s[0:2]) # "Ze"
```

2.2.5 Bytes

Byte literals use the `b""` prefix. Each element is a `u8` (0–255).

```
let data = b"\x48\x65\x6c\x6c\x6f"  # "Hello" as bytes
let empty = b""
```

Bytes support indexing (`data[0]` → `72` as int) and slicing. Bytes are *not* automatically convertible to strings; use the `string` module or explicit decoding.

2.2.6 Lists

List literals use square brackets. Heterogeneous contents are allowed.

```
let nums = [1, 2, 3]
let mixed = [1, "two", 3.0, true, nil]
let nested = [[1, 2], [3, 4]]
let empty = []
```

Lists are mutable, ordered, and allow duplicates. Indexing supports negative indices (from the end). Slicing `l[a:b]` returns a new list (copy, not a view).

```
let l = [10, 20, 30, 40, 50]
print(l[0])    # 10
print(l[-1])   # 50
print(l[1:4])  # [20, 30, 40]
l[2] = 99      # mutation
print(l)       # [10, 20, 99, 40, 50]
```

2.2.7 Dictionaries

Dict literals use `{key: value, ...}`. Keys must be strings. Values can be any type. Insertion order is preserved.

```
let person = {"name": "Zen", "version": 3, "active": true}
let nested = {"user": {"name": "Zen", "id": 1}, "tags": ["a", "b"]}
let empty = {}
```

Access `d["key"]`. Missing keys raise `KeyError`.

```
print(person["name"])   # Zen
person["version"] = 4   # update
person["lang"] = "Zen"  # add
print(person["missing"]) # KeyError
```

2.2.8 Sets

Set literals use `{value, value, ...}` with no colons. Elements must be hashable (strings, ints, floats, bools, tuples of hashables). Sets are unordered and deduplicate.

```
let s = {1, 2, 2, 3}
print(s)  # {1, 2, 3} — order not guaranteed, duplicates removed
print(2 in s)   # true
print(5 in s)   # false
```

Warning: `{}` alone is an empty *dict*, not an empty set. Use `set()` or `{1}` for a set.

2.2.9 Tuples

Tuple literals use `#(...)` with hash prefix. Immutable, ordered, heterogeneous.

```
let point = #(3, 4)
let pair = #("name", 42)
let single = #(5,)      # trailing comma required for 1-tuple
let empty = #()         # empty tuple
```

Access `t[0]`, `t[1]`. Tuples support destructuring in `match` patterns.

```
let p = #(10, 20)
print(p[0])   # 10
print(len(p)) # 2
```

2.3 2.3 Type coercion matrix

Zen does **not** implicitly coerce between most types. The `+` operator stringifies non-string operands when the other side is a string, but arithmetic and comparison require matching numeric types.

From → To	int	float	string	bool
int	identity	`42 → 42.0`	`"42"`	`0 → false`, else true
float	`3.9 → 3` (trunc)	identity	`"3.9"`	`0.0 → false`, else true
string	`"42" → 42` (parse)	`"3.14" → 3.14`	identity	`"" → false`, else true
bool	`true → 1`, `false → 0`	`true → 1.0`	`"true"/"false"`	identity

Explicit conversion functions:

```
int("42")       # 42
int(3.9)        # 3 (truncates toward zero)
float(42)       # 42.0
str(42)         # "42"
str(3.14)       # "3.14"
bool(0)         # false
bool(1)         # true
bool("")        # false
bool("hello")   # true
```

String-to-number parsing failures raise `ValueError`:

```
try {
    int("not a number")
} catch as e {
    print(e.msg)  # ValueError: invalid literal for int()
}
```

2.4 2.4 Truthiness

Zen truthiness: everything is truthy except `nil` and `false`.

Value	Truthy?
`nil`	false
`false`	false
`0`	**true** (unlike Python)
`0.0`	**true**
`""`	**true** (unlike Python)
`[]`	**true** (unlike Python)
`{}`	**true** (unlike Python)
`42`	true
`"hello"`	true
`[1, 2]`	true

Use `bool(x)` to coerce explicitly:

```
print(bool(0))    # true
print(bool(""))   # true
print(bool(nil))  # false
print(bool(false)) # false
```

2.5 2.5 Equality vs identity

`==` compares *value* (with numeric coercion: `1 == 1.0` is true). `is` compares *type membership* when the right side is a type name. There is no separate object-identity operator for heap objects beyond reference behavior; two equal lists are `==` but distinct objects.

```
print(1 == 1.0)   # true (numeric coercion)
print(1 == "1")   # false (type mismatch)
print([] == [])   # true (empty lists equal by value)
print({} == {})   # true (empty dicts equal by value)
```

Dict equality is order-insensitive:

```
print({"a": 1, "b": 2} == {"b": 2, "a": 1})  # true
```

2.6 2.6 Numeric edge cases

2.6.1 Infinity and NaN

```
let inf = 1e400          # +infinity
let neg_inf = -1e400     # −infinity
let nan = 0.0 * inf      # not-a-number

print(inf > 1e308)       # true
print(neg_inf < -1e308)  # true
print(nan == nan)        # false
print(nan != nan)        # true   ← NaN detection
print(inf + inf)         # inf
print(inf - inf)         # nan
print(0.0 * inf)         # nan
```

2.6.2 Integer overflow

```
let max = 9223372036854775807
print(max + 1)  # wraps to -9223372036854775808 (two's complement, release)
```

2.6.3 Division

```
print(7 / 2)    # 3.5 (float division)
print(7 // 2)   # 3 (floor division, integer result)
print(7 % 2)    # 1 (modulo, sign follows dividend)
print(-7 % 2)   # -1
print(-7 // 2)  # -4 (floor toward −inf)
```

Division by zero raises `DivideByZero`:

```
try {
    let x = 1 / 0
} catch as e {
    print(e.msg)  # DivideByZero: division by zero
}
```

2.7 2.7 Type conversion reference

```
# Number ↔ string
int("42")           # 42
int("3.9")          # 3 (truncates after float parse)
int(3.9)            # 3
float("3.14")       # 3.14
float(42)           # 42.0
str(42)             # "42"
```

```
str(3.14)           # "3.14"
str(true)           # "true"
str(nil)            # "nil"

# Boolean
bool(0)             # false
bool(1)             # true
bool(0.0)           # false
bool("")            # true
bool([])            # true
bool({})            # true
bool(nil)           # false
bool(false)         # false

# Collections
list("abc")         # ["a", "b", "c"]
tuple([1, 2])       # #(1, 2)
set([1, 1, 2])      # {1, 2}
dict([["a", 1]])    # {"a": 1}
```

2.8 2.8 Common pitfalls

- **`0` and `""` are truthy** in Zen (unlike Python). Use `x == 0` or `x == ""` for emptiness checks, not `if x`.
- **`nan != nan`**. Use `x != x` to detect NaN.
- **`{}` is an empty dict**, not a set. Use `{1}` or `set()` for sets.
- **Strings are immutable**. `s[0] = "X"` raises `TypeError`. Build new strings with concatenation.
- **Integer division `//` floors toward negative infinity**: `-7 // 2` is `-4`, not `-3`.
- **List slicing copies**: `l[1:3]` is a new list; mutating it doesn't affect the original.
- **Dict keys must be strings**: `d[42]` raises `TypeError`. Use `d["42"]`.

2.9 2.9 Quick reference

```
let x = 42
let f = 3.14
let s = "hello"
let b = true
let n = nil
let l = [1, 2, 3]
let d = {"a": 1}
let st = {1, 2}
let t = #(1, 2)

typeof(x)      # "int"
x is int       # true
```

```
x is string    # false
bool(x)        # true
str(x)         # "42"
int("42")      # 42
float(x)       # 42.0
len(l)         # 3
len(d)         # 1
len(s)         # 5
x == 42.0      # true
x == "42"      # false
```

2.10 2.10 Exercises

1. Write `is_nan(x)` that returns true only for NaN (hint: `x != x`).
2. Write `safe_int(s)` that parses a string to int, returning `nil` on failure instead of raising.
3. Demonstrate that `0` is truthy: print a message if `if 0 { ... }` executes.
4. Create a list, slice it, mutate the slice, and confirm the original is unchanged.
5. Show that `{"a": 1, "b": 2} == {"b": 2, "a": 1}` (dict order-insensitive equality).
6. Compute `-7 // 2` and `-7 % 2` and explain the results (floor division).
7. Write `type_name(x)` that returns a human-readable string for any value using `typeof()` and special-casing `nan`/`inf`.

3 Chapter 3: Operators and Expressions

Zen's operators cover arithmetic, comparison, logic, string handling, and safe member access. This chapter lists every operator, its precedence, associativity, and edge cases.

3.1 3.1 Operator precedence (high to low)

Precedence	Operator(s)	Associativity	Description
1	`() [] . ?.`	left	Grouping, indexing, field access, safe access
2	`**`	right	Power
3	`-` (unary) `not !`	right	Negation, logical not
4	`* / // %`	left	Multiply, divide, floor-divide, modulo
5	`+ -`	left	Add, subtract, concatenate
6	`== != < <= > >=`	left	Comparison
7	`and`	left	Logical and (short-circuits)
8	`or`	left	Logical or (short-circuits)
9	`??`	left	Nullish coalescing
10	`if-else` (ternary)	right	Conditional expression
11	`= += -= …`	right	Assignment

Use parentheses whenever precedence is unclear.

3.2 3.2 Arithmetic

Operator	Meaning	Example	Result	Notes
`+`	add	`1 + 2`	`3`	also string concat
`-`	subtract	`5 - 3`	`2`	
`*`	multiply	`3 * 2`	`6`	also string repeat `"x"*3`
`/`	divide (float)	`7 / 2`	`3.5`	div by zero → `DivideByZero`
`//`	floor divide	`7 // 2`	`3`	floors toward −inf

Operator	Meaning	Example	Result	Notes
`%`	modulo	`7 % 3`	`1`	sign follows dividend
`**`	power	`2 ** 3`	`8`	right-associative
`-`	negate	`-x`	`-x`	unary

3.2.1 Floor division and modulo edge cases

```
print(7 // 2)    # 3
print(-7 // 2)   # -4   (floor toward −inf, not truncation)
print(7 % 2)     # 1
print(-7 % 2)    # -1   (sign follows dividend)
print(7.5 // 2)  # 3.0  (floor works on floats too)
```

3.2.2 Division by zero

```
try {
    print(1 / 0)
} catch as e {
    print(e.msg)  # DivideByZero: division by zero
}
```

3.3 3.3 String concatenation and repetition

When either side of `+` is a string, the other side is stringified:

```
print("count: " + 42)       # "count: 42"
print("flag: " + true)      # "flag: true"
print(nil + "end")          # "nilend"
print("Zen " + "3.0")       # "Zen 3.0"
```

`42 + "count"` is a **type error** (number + string without string on the left). Use `str(42) + "count"`.

`*` repeats strings:

```
print("x" * 5)      # "xxxxx"
print("ab " * 3)    # "ab ab ab "
print(3 * "ab")     # also works: "abab"
```

3.4 3.4 Comparison

Operator	Meaning	Notes
`==`	equal	numeric coercion: `1 == 1.0` → true

Operator	Meaning	Notes
`!=`	not equal	
`<`	less than	
`<=`	less than or equal	
`>`	greater than	
`>=`	greater than or equal	

Numeric coercion in ==:

```
print(1 == 1.0)    # true
print(1 == "1")    # false (type mismatch, no coercion to string)
print([] == [])    # true (empty lists equal by value)
print({} == {})    # true (empty dicts equal, order-insensitive)
print({"a":1,"b":2} == {"b":2,"a":1})  # true
```

NaN: `nan == nan` is false; `nan != nan` is true.

3.5 3.5 Logic

Zen truthiness: everything is truthy except `nil` and `false`. Notably `0`, `0.0`, `""`, `[]`, `{}` are **truthy**.

Operator	Meaning	Short-circuits?
`and`	logical and	yes — returns first falsy or last
`or`	logical or	yes — returns first truthy or last
`not`	logical not	prefix, returns bool
`!`	logical not	alias for `not`

```
print(true and false)   # false
print(true or false)    # true
print(not true)         # false
print(!false)           # true

# Short-circuit: right side not evaluated if left decides
print(false and error())  # false — error() never called
print(true or error())    # true  — error() never called
```

`and`/`or` return the operand value, not necessarily a bool:

```
print("hello" and "world")   # "world"
print("" or "fallback")     # "fallback"  (note: "" is truthy in Zen!)
print(nil or "fallback")    # "fallback"
```

3.6 3.6 Safe member access: `?.` (optional chaining)

`?.` reads a field only if the object is non-nil; otherwise short-circuits to `nil`.

```
let user = nil
print(user?.name)           # nil  (no error)

let user = { name: "Zen" }
print(user?.name)           # "Zen"

let data = nil
print(data?.a?.b?.c)        # nil  (stops at first nil)
```

Compare with strict access, which raises on nil:

```
let user = nil
print(user.name)   # TypeError: cannot read field on nil
```

`?.` works on dicts, instances, and any object with fields. It does **not** work on lists for indexing — use `list[i]` directly or check bounds first.

3.7 3.7 Nullish coalescing: ??

`??` returns its left side if non-nil, otherwise the right side. It's a **nil-default**, not a falsy-default — `0`, `false`, and `""` are returned as-is.

```
print(nil ?? "default")     # "default"
print("set" ?? "default")   # "set"
print(0 ?? "default")       # 0      (0 is not nil)
print(false ?? "default")   # false
print(""  ?? "default")     # ""     (empty string is not nil)
print(nil ?? nil ?? "fb")   # "fb"   (chaining)
```

Common use: config fallback:

```
let timeout = config?.timeout ?? 30
```

3.8 3.8 Conditional expression (ternary)

```
let status = if score >= 60 { "pass" } else { "fail" }
```

The condition must be braced. The result is the value of the taken branch; both branches must be type-compatible (Zen is dynamically typed, so any values work, but mixing types may surprise downstream code).

3.9 3.9 Assignment and augmented assignment

```
let x = 10
x += 5      # x = 15
x -= 3      # x = 12
x *= 2      # x = 24
x /= 4      # x = 6.0  (float division)
```

```
x //= 2     # x = 3
x %= 2      # x = 1
```

Augmented assignment works on any lvalue (variable, field, index):

```
let d = {"count": 0}
d["count"] += 1   # d["count"] is now 1
```

3.10 3.10 `is` — type membership

`x is TypeName` checks whether `x`'s runtime type matches the named type:

```
print(42 is int)          # true
print(3.14 is float)      # true
print("hi" is string)     # true
print([] is list)         # true
print({} is dict)         # true
print(true is bool)       # true
print(nil is nil)         # true
print(42 is string)       # false
```

The right side must be a type keyword/literal, not a variable:

```
let t = "int"
print(42 is t)   # ParseError — type name required literally
```

Use `typeof(x) == t` instead for dynamic type-name comparison.

3.11 3.11 `in` / `!in` — membership

```
print(2 in [1, 2, 3])      # true
print(5 in [1, 2, 3])      # false
print(5 !in [1, 2, 3])     # true
print("a" in {"a": 1})     # true (dict key membership)
print("z" in "Zen")        # true (substring)
```

For strings, `in` checks substring membership. For lists/sets/dicts, it checks element/key membership.

3.12 3.12 Quick reference

```
# Arithmetic
1 + 2       # 3
7 / 2       # 3.5
7 // 2      # 3
7 % 3       # 1
2 ** 3      # 8
-5          # -5
```

```
# Strings
"a" + "b"   # "ab"
"x" * 3     # "xxx"
"count: " + 42   # "count: 42"

# Comparison
1 == 1.0    # true
"a" != "b"  # true

# Logic
true and false   # false
true or false    # true
not true         # false

# Safe access
nil?.name        # nil
{name: "Z"}?.name  # "Z"

# Coalescing
nil ?? "d"       # "d"
0 ?? "d"         # 0

# Ternary
if ok { "yes" } else { "no" }

# Type check
42 is int        # true

# Membership
2 in [1,2,3]     # true
"a" in "Zen"     # true
```

3.13 3.13 Exercises

1. Write `sign(x)` returning `1`, `-1`, or `0` using only comparison operators.
2. Demonstrate that `0` is truthy: `if 0 { print("zero is truthy") }` prints.
3. Write `safe_div(a, b)` returning `a // b` or `nil` if `b == 0`, using `try`/`catch`.
4. Show that `"0" ?? "default"` returns `"0"` (string “0” is not nil).
5. Prove `nan != nan` using `0.0 * inf`.
6. Write `in_range(x, lo, hi)` using chained comparisons (Zen does **not** chain `lo < x < hi` — use `lo < x and x < hi`).
7. Show the difference between `and` short-circuiting: `false and error()` does not evaluate `error()`.

4 Chapter 5: Functions

Functions are first-class values in Zen. They can be declared with `fn`, assigned to variables, passed as arguments, returned from other functions, and stored in collections. This chapter covers declaration, calling, closures, default arguments, variadic behavior, and the `return` statement.

4.1 5.1 Declaration and calling

```
func add(a, b) {
    return a + b
}

print(add(2, 3))   # 5
```

- `func name(params) { body }` declares a function.
- The body is a block of statements; the last expression is **not** automatically returned — use `return`.
- Functions are values: `let f = add` then `f(2, 3)` works.

4.1.1 Calling with wrong argument count

```
try {
    add(1)            # missing argument
} catch as e {
    print(e.msg)      # ArgumentCount: expected 2, got 1
}

try {
    add(1, 2, 3)      # too many
} catch as e {
    print(e.msg)      # ArgumentCount: expected 2, got 3
}
```

Zen does **not** have keyword arguments. All calls are positional.

4.2 5.2 Return statements

`return` exits the function immediately with a value:

```
func abs(x) {
    if x < 0 {
        return -x
    }
    return x
}
```

```
print(abs(-5))  # 5
print(abs(3))   # 3
```

A function without `return` returns `nil`:

```
func nop() {
    print("side effect only")
}
print(nop())    # nil
```

`return` inside nested blocks exits the **enclosing function**, not just the block:

```
func find(items, target) {
    for i in 0..len(items) {
        if items[i] == target {
            return i          # exits find(), not just the if
        }
    }
    return -1
}
```

4.3 5.3 Default arguments

Zen supports default values for trailing parameters:

```
func greet(name, greeting = "Hello") {
    return greeting + ", " + name + "!"
}

print(greet("Zen"))            # "Hello, Zen!"
print(greet("Zen", "Hi"))      # "Hi, Zen!"
```

Rules: - Defaults must be on trailing parameters (no required parameter after a defaulted one). - Default expressions are evaluated at call time, not definition time: `zen let counter = 0 func next(n = counter + 1) { counter = counter + 1 return n } print(next()) # 1 print(next()) # 2` (counter incremented between calls)

4.4 5.4 Variadic behavior

Zen does **not** have `*args` or `**kwargs` syntax. All parameters are fixed-position. However, you can pass a list and unpack it manually:

```
func sum_all(nums) {
    let total = 0
    for n in nums {
        total = total + n
    }
    return total
}
```

```
let vals = [1, 2, 3, 4]
print(sum_all(vals))   # 10
```

If you need to forward arguments to another function, pass them explicitly or as a list.

4.5 5.5 Functions as first-class values

```
let f = add
print(f(10, 20))   # 30

let ops = [add, sub, mul]
print(ops[0](5, 3))  # 8
```

You can store functions in lists, dicts, and pass them to other functions:

```
func apply(f, x, y) {
    return f(x, y)
}

print(apply(add, 3, 4))   # 7
print(apply(sub, 3, 4))   # -1
```

4.6 5.6 Closures

Functions capture variables from their enclosing scope. Captured variables are shared (boxed) cells, so mutations are visible across calls:

```
func make_counter(start) {
    var count = start
    func inc() {
        count = count + 1
        return count
    }
    return inc
}

let c1 = make_counter(0)
let c2 = make_counter(100)

print(c1())   # 1
print(c1())   # 2
print(c2())   # 101
print(c1())   # 3   (c1 and c2 have independent count cells)
```

Each call to `make_counter` creates a **new** `count` cell. `c1` and `c2` capture different cells.

Closure over loop variables — be careful:

```
let funcs = []
for i in 1..3 {
    funcs.push(fn() { return i })
}
print(funcs[0]())   # 3 (all capture the same final i)
print(funcs[1]())   # 3
print(funcs[2]())   # 3
```

To capture per-iteration values, use a helper:

```
func make_fn(val) {
    return fn() { return val }
}

let funcs = []
for i in 1..3 {
    funcs.push(make_fn(i))
}
print(funcs[0]())   # 1
print(funcs[1]())   # 2
print(funcs[2]())   # 3
```

4.7 5.7 Higher-order functions

```
func twice(f, x) {
    return f(f(x))
}

func add_one(x) {
    return x + 1
}

print(twice(add_one, 5))   # 7

func map_list(f, items) {
    let result = []
    for x in items {
        result.push(f(x))
    }
    return result
}

print(map_list(add_one, [1, 2, 3]))   # [2, 3, 4]
```

4.8 5.8 Anonymous functions (lambdas)

Zen supports `lambda(params) { body }`:

```
let double = lambda(x) { return x * 2 }
print(double(21))   # 42
```

Lambdas are functions too — first-class, closures, passable. The difference from `fn` is syntactic: `lambda` is an expression, `fn` is a statement.

```
# lambda as expression (inline)
let result = (lambda(x, y) { return x + y })(3, 4)
print(result)   # 7
```

4.9 5.9 `return` in closures

`return` inside a lambda returns from the lambda, not the enclosing function:

```
func outer() {
    let f = lambda(x) {
        if x < 0 { return -1 }   # returns from the lambda
        return x * 2
    }
    return f
}

let f = outer()
print(f(-5))   # -1
print(f(5))    # 10
```

4.10 5.10 Common pitfalls

- **No keyword arguments.** All calls are positional. `add(a=1, b=2)` is a parse error.
- **Default args are evaluated at call time.** Mutable default expressions can surprise.
- **`return` exits the enclosing function**, not just the current block.
- **Closures capture by reference (shared cell)**, not by value. Mutating a captured var affects all closures over it.
- **Loop variable capture**: all closures in a loop share the loop var unless you use a helper function to bind per-iteration values.
- **Functions without `return` return `nil`.**

4.11 5.11 Quick reference

```
func name(a, b = default) { ... return value }
let f = name
f(1, 2)

lambda(x) { return x * 2 }
lambda(x, y) { return x + y }(3, 4)   # inline call
```

```
func make_counter(start) {
    var count = start
    return fn() { count = count + 1; return count }
}

func apply(f, x, y) { return f(x, y) }

return               # returns nil
return value         # returns value
```

4.12 5.12 Exercises

1. Write `compose(f, g)` that returns a function `h(x) = f(g(x))`.
2. Write `repeater(n)` that returns a function which, when called, returns the call count (1, 2, 3, ...).
3. Write `once(f)` that returns a wrapper calling `f` only on the first invocation and caching the result.
4. Write `curry2(f)` that takes a two-arg function and returns a function taking one arg that returns a function taking the second.
5. Write `memoize(f)` that caches results by argument (assume a single hashable arg).
6. Fix the loop-variable capture bug: create 3 functions returning 1, 2, 3 respectively using a helper.
7. Write `pipeline(funcs, x)` that applies each function in `funcs` to `x` in sequence.

5 Chapter 6: Lambdas and Closures

This chapter is a deep dive into anonymous functions and closure semantics: how captures work, what is shared vs copied, the `lambda` expression vs `fn` statement, and how closures interact with the JIT and reference counting.

5.1 6.1 Lambda expressions

A lambda is an anonymous function defined as an expression:

```
let double = lambda(x) { return x * 2 }
print(double(21))   # 42
```

Syntax: `lambda(params) { body }`. The body is a block; use `return` to produce a value (omitting `return` yields `nil`).

5.1.1 Lambda as expression vs fn as statement

	`lambda`	`fn`
Kind	Expression	Statement
Can appear inline	Yes (`(lambda(x){...})(1)`)	No
Assignable	Yes	Yes (`let f = fn() {...}`)
First-class	Yes	Yes
Closure capture	Yes	Yes
`return` target	The lambda itself	The enclosing function

Because `lambda` is an expression, you can call it immediately:

```
let result = (lambda(x, y) { return x + y })(3, 4)
print(result)   # 7
```

And pass it directly:

```
print((lambda(x) { return x * 2 })(5))   # 10
```

5.2 6.2 Closure capture semantics

Zen functions (both `fn` and `lambda`) capture variables from their enclosing scope by **shared reference** (a boxed `cell`). Mutating a captured variable is visible to every closure that captured it, and persists across calls.

5.2.1 Basic capture

```
func make_counter(start) {
    var count = start
    func inc() {
        count = count + 1
```

```
        return count
    }
    return inc
}

let c1 = make_counter(0)
let c2 = make_counter(100)

print(c1())   # 1
print(c1())   # 2
print(c2())   # 101
print(c1())   # 3   (c1 and c2 are independent)
```

Each call to `make_counter` allocates a fresh `count` cell. `c1` and `c2` point to different cells, so they count independently.

5.2.2 Multiple closures share the same cell

```
func make_pair() {
    var value = 0
    func get() { return value }
    func set(v) { value = v }
    return #(get, set)
}

let #(get, set) = make_pair()
print(get())     # 0
set(42)
print(get())     # 42
```

`get` and `set` share the same `value` cell — mutation through one is visible through the other.

5.3 6.3 The loop-variable capture trap

This is the most common closure bug. A loop variable is a single cell; every closure created in the loop captures **that same cell**, so they all see the final value:

```
let funcs = []
for i in 1..3 {
    funcs.push(fn() { return i })
}
print(funcs[0]())   # 3   ← not 1!
print(funcs[1]())   # 3
print(funcs[2]())   # 3
```

All three closures return `3` because `i` ended at `3` (the loop exits when `i == 3`).

5.3.1 Fix: bind per-iteration with a helper function

```
func make_fn(val) {
    return fn() { return val }
}

let funcs = []
for i in 1..3 {
    funcs.push(make_fn(i))   # i is copied into `val` at each call
}
print(funcs[0]())   # 1
print(funcs[1]())   # 2
print(funcs[2]())   # 3
```

`make_fn(i)` copies the current `i` into a fresh `val` parameter cell, so each closure captures its own `val`.

5.4 6.4 `return` target in closures

`return` inside a lambda returns from the lambda, **not** from the enclosing function:

```
func outer() {
    let f = lambda(x) {
        if x < 0 { return -1 }   # returns from lambda
        return x * 2
    }
    return f
}

let f = outer()
print(f(-5))   # -1
print(f(5))    # 10
print(outer()(-3))  # -1
```

If you need the enclosing function to return from inside a closure, use an error or a sentinel value instead.

5.5 6.5 Closures and reference counting

A closure retains every variable it captures. As long as the closure is alive, its captured variables stay alive:

```
func make_big() {
    let big = [0 for _ in 1..1000000]   # large allocation
    return fn() { return len(big) }     # keeps `big` alive
}

let f = make_big()
print(f())   # 1000000   (big still in memory)
```

When `f` is dropped (goes out of scope or reassigned), the captured cell is released, and `big` can be collected.

```
f = nil
# now `big` can be freed
```

5.6 6.6 Closures in collections

Functions are first-class, so they can live in lists and dicts:

```
let ops = {
    "add": lambda(a, b) { return a + b },
    "sub": lambda(a, b) { return a - b },
}

print(ops["add"](5, 3))   # 8
print(ops["sub"](5, 3))   # 2
```

5.7 6.7 Closures and threads

Closures can be passed to `thread.spawn`, but captured shared variables become concurrently accessible — use `shared` for atomic counters:

```
import shared
import thread

let counter = shared.create(0)

for i in 1..3 {
    thread.spawn(fn() {
        shared.increment(counter, 1)
    })
}
# ... join threads ...
print(shared.get(counter))   # 3
```

Without `shared`, each thread would capture its own copy and updates would be lost.

5.8 6.8 JIT lowering note

Zen's Cranelift JIT lowers hot closures to native code. Closures that are called repeatedly in a loop get tiered up; the capture cells are accessed through a stable context pointer. You generally don't need to think about this, but it means:

- A closure called once stays interpreted.
- A closure called millions of times becomes native code with equivalent capture semantics.
- `ZEN_NO_JIT=1` disables tiering (all interpreted, same semantics).

5.9 6.9 Quick reference

```
# Lambda expression
let f = lambda(x) { return x * 2 }
print(f(5))           # 10

# Inline call
print((lambda(x) { return x + 1 })(41))   # 42

# Closure with shared cell
func make_counter(start) {
    var count = start
    return fn() { count = count + 1; return count }
}

# Fix loop capture
let funcs = []
for i in 1..3 {
    funcs.push((lambda(v) { return fn() { return v } })(i))
}
print(funcs[0]())   # 1

# return from lambda, not enclosing func
let f = fn(x) { if x < 0 { return -1 } return x * 2 }
```

5.10 6.10 Exercises

1. Write `make_adder(n)` returning a lambda that adds `n` to its argument.
2. Write `make_accumulator()` returning `fn(x)` that adds `x` to a running total and returns the total.
3. Write `repeater(n)` returning `fn()` that returns `n` on every call, but after `k` calls returns `nil` (capture a counter).
4. Write `compose(f, g)` returning `fn(x) { return f(g(x)) }`.
5. Demonstrate the loop-capture bug and fix it with `make_fn`.
6. Write `twice(f)` returning `fn(x) { return f(f(x)) }`.
7. Write `curry(f)` for a two-argument function: `curry(f)(a)(b)` equals `f(a, b)`.

6 Chapter 11: Error Handling

Zen is "safe by default": runtime problems become catchable errors instead of crashes. This chapter covers `try`/`catch`/`throw`/`finally` exhaustively, error fields, typed catch clauses, error types, and the interaction between `exit()` and `finally`.

6.1 11.1 Catching errors: `try` / `catch`

Wrap risky code in `try`. If anything inside raises an error, execution jumps to the matching `catch` block.

```
try {
    let x = 1 / 0        # raises DivideByZero
    print("never reached")
} catch {
    print("division failed")
}
# prints: division failed
```

6.1.1 Bare `catch`

A bare `catch` handles **any** error. If the `try` body completes normally, `catch` is skipped:

```
var caught = false
try {
    var bad = 1 / 0
    caught = bad == 0
} catch {
    caught = true
}
print(caught)  # true

var ok = true
try {
    var five = 10 / 2
    ok = five == 5
} catch {
    ok = false
}
print(ok)  # true
```

6.2 11.2 Reading the real failure: `catch as e`

Without a binding, you only know *that* something failed, not *what*. Bind the error with `catch as e` to inspect it:

```
try {
    let y = 1 / 0
} catch as e {
```

```
    print(str(e))   # DivideByZero: division by zero
}
```

`str(e)` renders the real message. You can also concatenate a string with the error directly:

```
try {
    let y = 1 / 0
} catch as e {
    print("caught: " + e)   # caught: DivideByZero: division by zero
}
```

6.3 11.3 Error fields: `e.msg`, `e.code`, `e.val`

A caught error is an object with three readable fields:

Field	Meaning	Example
`e.msg`	Human-readable message	`"DivideByZero: division by zero"`
`e.code`	Numeric error code	`0`
`e.val`	Extra attached value (often `nil`)	`nil`

```
try {
    let a = [1][5]   # IndexError
} catch as e {
    print(e.msg)   # IndexError: index 5 out of bounds for length 1
    print(e.code)  # 0
    print(e.val)   # nil
}
```

6.4 11.4 String + error concatenation

```
try {
    let y = 1 / 0
} catch as e {
    print("caught: " + e)   # caught: DivideByZero: division by zero
}
```

6.5 11.5 Catching specific error types

You can add typed clauses to handle only certain errors. Unmatched clauses fall through to the next one:

```
try {
    risky()
```

```
} catch TypeError {
    print("bad type")
} catch UserError {
    print("user raised")
} catch {
    print("everything else")
}
```

A clause whose type does not match lets the error keep propagating outward:

```
try {
    throw "z"
} catch TypeError {   # doesn't match UserError
    print("t")        # skipped
} catch UserError {
    print("u")        # catches it
}
print("u")  # u
```

6.6 11.6 Raising errors: `throw`

Use `throw` to raise your own errors. Strings become `UserError` messages:

```
var msg = ""
try {
    throw "custom failure"
} catch {
    msg = "handled"
}
print(msg)  # handled

var tm = ""
try {
    throw "custom boom"
} catch as e {
    tm = e.msg
}
print(tm)  # UserError: custom boom
```

`throw` accepts any value:

```
throw { code: 42, detail: "retry later" }
```

The value is stringified into `e.msg`; `e.val` is `nil` for `throw`:

```
try {
    throw { code: 42 }
} catch as e {
    print(e.msg)   # UserError: { code: 42, detail: ... }
```

```
    print(e.val)   # nil
}
```

6.7 11.7 Cleanup: `finally`

A `finally` block runs whether the `try` succeeded or failed, and whether or not `catch` handled the error. It must come **after** any `catch` clauses.

```
var log = ""
try {
    throw "boom"
} catch {
    log = log + "c"
} finally {
    log = log + "f"
}
print(log)  # cf
```

```
# finally also runs when nothing failed
var log2 = ""
try {
    log2 = "b"
} finally {
    log2 = log2 + "f"
}
print(log2)  # bf
```

6.8 11.8 `exit()` and `finally`

`exit(code)` terminates the process immediately with the given code. Because `exit` terminates the process, `finally` blocks do **not** run after `exit()`:

```
try {
    exit(3)
} finally {
    print("this will NOT print")   # process already exited
}
```

```
# exit inside catch does not run finally
try {
    throw "boom"
} catch {
    exit(1)
} finally {
    print("this will NOT print")   # process exited in catch
}
```

If you need cleanup before exiting, do it before calling `exit()`:

```
try {
    risky()
} catch as e {
    cleanup_resources()   # explicit cleanup before exit
    exit(1)
}
```

6.9 11.9 Nested handlers

Inner handlers run before outer ones:

```
var order = ""
try {
    try { throw "inner" } catch { order = order + "i" }
    throw "outer"
} catch { order = order + "o" }
print(order)  # io
```

6.10 11.10 Errors from callees

Errors raised inside a function are catchable by the caller:

```
func deep() {
    return [1][5]
}
var hit9 = false
try {
    deep()
} catch IndexError {
    hit9 = true
}
print(hit9)  # true
```

6.11 11.11 Error types

Zen's built-in error types:

Type	Meaning	Typical cause
`TypeError`	Wrong type for an operation	`"a" + 1`, `nil.field`
`IndexError`	Sequence index out of bounds	`[1][5]`, `"ab"[5]`
`KeyError`	Missing dict key	`d["missing"]`
`DivideByZero`	Division by zero	`1 / 0`, `1 % 0`
`ValueError`	Inappropriate value	`int("not a number")`
`NativeError`	Raised by a native module	fs, socket, etc.

Type	Meaning	Typical cause
`ForeignAttribute`	Failed foreign attribute access	`py_obj.bad_attr`
`PyImportError`	Python import failure	`pymport nonexistent`
`UserError`	Raised by `throw`	`throw "msg"`

6.12 11.12 Full error-code reference

Every error carries a code `Z####`. See Appendix B for the complete list. Common runtime codes:

Code	Type	Meaning
`Z0401`	TypeError	Type mismatch
`Z0402`	IndexError	Index out of bounds
`Z0403`	KeyError	Key not found
`Z0404`	DivideByZero	Division by zero
`Z0405`	ValueError	Value error
`Z0406`	Attribute error	Attribute access
`Z0407`	NativeError	Native module error
`Z0408`	ArgumentCount	Wrong arg count
`Z0409`	HandleError	Handle error
`Z0410`	IOMismatch	I/O mismatch
`Z0411`	NativeError	Native module error (general)
`Z0414`	PyImportError	Python import failed

Use `zen explain Z0401` to see the full explanation for any code.

6.13 11.13 Common pitfalls

- **`e` only exists inside its `catch` block.** Don't reference it outside.
- **`finally` goes after `catch`, not between `try` and `catch`.**
- **A bare `catch` swallows everything.** Use typed clauses when you want unexpected errors to propagate.
- **`str(e)` is the readable message.** Never rely on `typeof(e)` (it's just `"error"`) — use `e.msg` instead.
- **`exit()` skips `finally`**. Do cleanup before calling `exit()`.
- **`throw` with a string** creates `UserError: <message>`; `e.val` is `nil`.
- **`throw` with an object** stringifies it into `e.msg`; `e.val` is `nil`.

6.14 11.14 Quick reference

```
try {
    # risky code
} catch as e {
    print(str(e))   # real message
```

```
    print(e.msg)    # same as str(e)
    print(e.code)   # numeric code
    print(e.val)    # attached value
} catch TypeError {
    # only type errors
} catch UserError {
    # only user-thrown errors
} catch {
    # everything else
} finally {
    # always runs (unless exit() was called)
}

throw "something went wrong"   # raise your own error
throw { code: 42 }             # throw any value

try {
    exit(1)   # process exits; finally does NOT run
}
```

6.15 11.15 Exercises

1. Write `safe_divide(a, b)` that returns `nil` on division by zero instead of crashing.
2. Use `try`/`catch` to read a file that might not exist, returning a default value.
3. Write a function that uses `try`/`catch`/`finally` to guarantee a cleanup message prints regardless of success or failure.
4. Demonstrate that `exit()` skips `finally`.
5. Write `parse_int_or_default(s, default)` that uses `try`/`catch` around `int(s)`.
6. Create a custom error type by throwing an object and inspecting `e.msg`.
7. Write a nested try/catch where the inner catch handles one error type and the outer catch handles another.

7 Chapter 25: crypto Module — Complete Guide

The `crypto` module provides symmetric encryption, asymmetric keys, hashing, HMAC, key derivation, JWT, OTP, and X.509. This chapter covers the complete crypto API.

> ⚠ Warning: Cryptography is easy to get wrong. Use well-established algorithms and modes. Never roll your own crypto for production security.

7.1 25.1 Symmetric encryption

```
import crypto

# Generate a key
let key = crypto.generate_key("aes-256-gcm")

# Encrypt
let ciphertext = crypto.encrypt_aes_gcm(key, "secret message")

# Decrypt
let plaintext = crypto.decrypt_aes_gcm(key, ciphertext)
print(plaintext)  # secret message
```

7.1.1 Symmetric functions

Function	Description
`generate_key(alg?)`	Generate a key (“aes-256-gcm” or “chacha20-poly1305”)
`encrypt(alg, key, plaintext, aad?)`	Encrypt with given algorithm
`decrypt(alg, key, ciphertext, aad?)`	Decrypt with given algorithm
`encrypt_aes_gcm(key, plaintext, aad?)`	AES-256-GCM encrypt
`decrypt_aes_gcm(key, ciphertext, aad?)`	AES-256-GCM decrypt
`encrypt_chacha20_poly1305(key, plaintext, aad?)`	ChaCha20-Poly1305 encrypt
`decrypt_chacha20_poly1305(key, ciphertext, aad?)`	ChaCha20-Poly1305 decrypt

```
import crypto

let key = crypto.generate_key("aes-256-gcm")
let ct = crypto.encrypt_aes_gcm(key, "hello world")
let pt = crypto.decrypt_aes_gcm(key, ct)
print(pt)  # hello world
```

```
let key2 = crypto.generate_key("chacha20-poly1305")
let ct2 = crypto.encrypt_chacha20_poly1305(key2, "secret")
let pt2 = crypto.decrypt_chacha20_poly1305(key2, ct2)
print(pt2)  # secret
```

7.1.2 With associated data (AAD)

AAD is authenticated but not encrypted. If you tamper with the ciphertext or AAD, decryption raises an error:

```
import crypto

let key = crypto.generate_key("aes-256-gcm")
let aad = "header-data"

let ct = crypto.encrypt_aes_gcm(key, "secret", aad)
let pt = crypto.decrypt_aes_gcm(key, ct, aad)
print(pt)  # secret

# Tampered AAD fails
try {
    crypto.decrypt_aes_gcm(key, ct, "tampered-aad")
} catch as e {
    print(e.msg)  # authentication failed
}
```

7.1.3 Generic encrypt/decrypt

```
import crypto

let key = crypto.generate_key("aes-256-gcm")

let ct = crypto.encrypt("aes-256-gcm", key, "plaintext")
let pt = crypto.decrypt("aes-256-gcm", key, ct)
print(pt)  # plaintext
```

7.2 25.2 Asymmetric encryption (RSA)

```
import crypto

let keys = crypto.generate_rsa(2048)
let public_key = keys.public_key
let private_key = keys.private_key

let ciphertext = crypto.rsa_encrypt(public_key, "secret")
```

```
let plaintext = crypto.rsa_decrypt(private_key, ciphertext)
print(plaintext)  # secret
```

7.2.1 Asymmetric functions

Function	Description
`generate_rsa(bits?)`	Generate RSA keypair (default 2048)
`rsa_encrypt(public_key, plaintext)`	RSA encrypt
`rsa_decrypt(private_key, ciphertext)`	RSA decrypt
`rsa_sign(private_key, message)`	RSA sign
`rsa_verify(public_key, message, sig)`	RSA verify → bool

```
import crypto

let keys = crypto.generate_rsa(2048)

print(keys.public_key)   # PEM public key
print(keys.private_key)  # PEM private key

let ct = crypto.rsa_encrypt(keys.public_key, "top secret")
let pt = crypto.rsa_decrypt(keys.private_key, ct)
print(pt)  # top secret
```

RSA encryption has size limits (key size / padding overhead). For large data, use hybrid encryption: encrypt data with a random symmetric key, encrypt the key with RSA.

7.3 25.3 Digital signatures

7.3.1 RSA sign/verify

```
import crypto

let keys = crypto.generate_rsa(2048)
let message = "hello world"

let sig = crypto.rsa_sign(keys.private_key, message)
let valid = crypto.rsa_verify(keys.public_key, message, sig)
print(valid)  # true

# Tampered message fails
let invalid = crypto.rsa_verify(keys.public_key, "tampered", sig)
print(invalid)  # false
```

7.3.2 Ed25519 sign/verify

```
import crypto

let keys = crypto.generate_ed25519()
let message = "hello world"

let sig = crypto.ed25519_sign(keys.private_key, message)
let valid = crypto.ed25519_verify(keys.public_key, message, sig)
print(valid)  # true

# Tampered fails
let invalid = crypto.ed25519_verify(keys.public_key, "tampered", sig)
print(invalid)  # false
```

7.3.3 Signature functions

Function	Description
`rsa_sign(private_key, message)`	RSA sign → signature
`rsa_verify(public_key, message, sig)`	RSA verify → bool
`ed25519_sign(private_key, message)`	Ed25519 sign → signature
`ed25519_verify(public_key, message, sig)`	Ed25519 verify → bool

```
import crypto

let keys = crypto.generate_ed25519()
let msg = "important document"

let sig = crypto.ed25519_sign(keys.private_key, msg)

# Anyone with the public key can verify
let ok1 = crypto.ed25519_verify(keys.public_key, msg, sig)
print(ok1)  # true

let ok2 = crypto.ed25519_verify(keys.public_key, "tampered", sig)
print(ok2)  # false
```

7.4 25.4 Key agreement (X25519)

```
import crypto

let alice = crypto.generate_x25519()
let bob = crypto.generate_x25519()

let alice_secret = crypto.x25519_shared_secret(alice.private_key, bob.public_key)
```

```
let bob_secret = crypto.x25519_shared_secret(bob.private_key, alice.public_key)

print(alice_secret == bob_secret)  # true (shared secret matches)
```

7.4.1 X25519 functions

Function	Description
`generate_x25519()`	Generate X25519 keypair
`x25519_shared_secret(private_key, public_key)`	Compute shared secret

```
import crypto

# Two parties establish a shared secret without transmitting it
let alice = crypto.generate_x25519()
let bob = crypto.generate_x25519()

// Alice computes: her private key + Bob's public key
let alice_shared = crypto.x25519_shared_secret(alice.private_key, bob.public_key)
// Bob computes: his private key + Alice's public key
let bob_shared = crypto.x25519_shared_secret(bob.private_key, alice.public_key)

print(alice_shared == bob_shared)  # true
```

7.5 25.5 Hashing and HMAC

```
import crypto

print(crypto.hash("sha256", "hello"))
print(crypto.hmac("sha256", "key", "message"))
```

Hash algorithms: `md5`, `sha1`, `sha224`, `sha256`, `sha384`, `sha512`, `sha3_256`, `sha3_512`, `blake2b`, `blake2s`.

HMAC algorithms: `sha256`, `sha512`.

```
import crypto

print(crypto.hash("sha256", "hello"))     # 2cf24dba...
print(crypto.hash("sha512", "hello"))     # longer hex
print(crypto.hash("blake2b", "hello"))    # BLAKE2b
print(crypto.hash("md5", "hello"))        # MD5 (broken, don't use for security)

let mac = crypto.hmac("sha256", "secret-key", "message")
print(mac)  # HMAC-SHA256 hex
```

```
let mac2 = crypto.hmac("sha512", "secret-key", "message")
print(mac2)  # HMAC-SHA512 hex
```

HMAC for message authentication:

```
import crypto

let key = "shared-secret"
let message = "authenticate this"

let mac = crypto.hmac("sha256", key, message)

# Receiver with same key verifies
let expected = crypto.hmac("sha256", key, message)
print(mac == expected)  # true

let wrong = crypto.hmac("sha256", "wrong-key", message)
print(mac == wrong)  # false
```

7.6 25.6 Key derivation (PBKDF2)

```
import crypto

let key = crypto.pbkdf2("password", "salt", 100000, 32)
print(key)  # derived key (hex)
```

7.6.1 PBKDF2

```
import crypto

# Derive a key from a password
let derived = crypto.pbkdf2("my-password", "random-salt", 100000, 32)
print(derived)  # 64 hex chars (32 bytes)

# More iterations = slower but more secure
let slow = crypto.pbkdf2("password", "salt", 1000000, 32)
```

Use PBKDF2 for password hashing (though Argon2/scrypt are preferred in modern systems). Always use a unique random salt per password.

7.7 25.7 JWT (JSON Web Tokens)

```
import crypto

let payload = {"sub": "123", "name": "Zen"}
let secret = "my-secret"
```

```
let token = crypto.jwt_encode(payload, secret)
print(token)  # eyJ...

let decoded = crypto.jwt_decode(token, secret)
print(decoded["sub"])  # 123
```

7.7.1 JWT functions

Function	Description
`jwt_encode(payload, secret)`	Create JWT
`jwt_decode(token, secret)`	Verify and decode JWT

```
import crypto

let payload = {"sub": "user-42", "role": "admin", "exp": 9999999999}
let secret = "super-secret"

let token = crypto.jwt_encode(payload, secret)
print(token)  # eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

let decoded = crypto.jwt_decode(token, secret)
print(decoded["sub"])    # user-42
print(decoded["role"])   # admin

# Tampered token fails
try {
    crypto.jwt_decode(token, "wrong-secret")
} catch as e {
    print(e.msg)  # signature verification failed
}
```

7.8 25.8 OTP (TOTP/HOTP)

```
import crypto

let totp = crypto.otp_totp("secret")
print(totp)  # 6-digit code

let valid = crypto.otp_verify("secret", "123456")
print(valid)  # true/false
```

7.8.1 OTP functions

Function	Description
`otp_totp(secret)`	Generate TOTP code
`otp_verify(secret, code)`	Verify TOTP code → bool

```
import crypto

let secret = "JBSWY3DPEHPK3PXP"

let code = crypto.otp_totp(secret)
print(code)  # 6-digit time-based code

let valid = crypto.otp_verify(secret, code)
print(valid)  # true (if within time window)

let invalid = crypto.otp_verify(secret, "000000")
print(invalid)  # false
```

TOTP codes change every 30 seconds. `otp_verify` accepts codes within a small time window (±1 step) to account for clock skew.

7.9 25.9 X.509 certificates

```
import crypto

let cert = crypto.x509_parse(cert_pem)
print(cert.subject)
print(cert.issuer)
print(cert.not_after)
```

7.9.1 X.509 functions

Function	Description
`x509_parse(pem)`	Parse PEM certificate

```
import crypto

let pem = "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----"

let cert = crypto.x509_parse(pem)
print(cert.subject)    # subject DN
print(cert.issuer)     # issuer DN
print(cert.not_after)  # expiry
print(cert.not_before) # validity start
```

7.10 25.10 Constant-time compare and secure zero

```
import crypto

# Timing-safe comparison (for HMACs, tokens, passwords)
let ok = crypto.constant_time_eq(expected, received)
print(ok)  # true/false

# Zeroize a secret after use
let pass = read_secret()
# ... use pass ...
crypto.secure_zero(pass)
pass = nil
```

```
import crypto

// Constant-time comparison prevents timing attacks
let expected = compute_hmac(key, message)
let received = hmac_from_request()

if crypto.constant_time_eq(expected, received) {
    print("authenticated")
} else {
    print("rejected")
}

// Secure zero - overwrite secret in memory
let password = read_user_password()
authenticate(password)
crypto.secure_zero(password)
password = nil
```

7.11 25.11 Algorithm policy (FIPS-ish mode)

```
import crypto

crypto.policy("strict")   # block md5, sha1, weak ciphers
crypto.policy("default")  # allow all
```

Or set `ZEN_FIPS=1` in the environment.

```
import crypto

// Check current policy behavior
try {
    crypto.hash("md5", "data")  // raises if strict
} catch as e {
    print(e.msg)  // blocked by crypto policy
```

```
}

crypto.policy("strict")
try {
    crypto.hash("md5", "data")  // still blocked
} catch as e {
    print("blocked as expected")
}

crypto.policy("default")
print(crypto.hash("md5", "data"))  # works again
```

7.11.1 Blocked algorithms in strict mode

Algorithm	Blocked in strict?
md5	yes
sha1	yes
sha224	yes
sha256	no
sha384	no
sha512	no
sha3_256	no
sha3_512	no
blake2b	no
blake2s	no
des	yes
rc4	yes

7.12 25.12 Quick reference

```
import crypto

# Symmetric
let key = crypto.generate_key("aes-256-gcm")
let ct = crypto.encrypt_aes_gcm(key, "secret")
let pt = crypto.decrypt_aes_gcm(key, ct)

# Asymmetric
let keys = crypto.generate_rsa(2048)
let ct = crypto.rsa_encrypt(keys.public_key, "secret")
let pt = crypto.rsa_decrypt(keys.private_key, ct)

# Signatures
let sig = crypto.ed25519_sign(keys.private_key, "msg")
let ok = crypto.ed25519_verify(keys.public_key, "msg", sig)
```

```
# Key agreement
let shared = crypto.x25519_shared_secret(a.private_key, b.public_key)

# Hashing
print(crypto.hash("sha256", "hello"))
print(crypto.hmac("sha256", "key", "msg"))

# KDF
let derived = crypto.pbkdf2("password", "salt", 100000, 32)

# JWT
let token = crypto.jwt_encode({"sub": "1"}, "secret")
let decoded = crypto.jwt_decode(token, "secret")

# Security utils
crypto.constant_time_eq(a, b)
crypto.secure_zero(secret)
crypto.policy("strict")
```

7.13 25.13 Exercises

1. Encrypt and decrypt a message with AES-256-GCM.
2. Generate an Ed25519 keypair, sign a message, and verify it.
3. Perform X25519 key agreement between two parties and confirm shared secrets match.
4. Create a JWT, decode it, and verify tampering fails.
5. Generate a TOTP code and verify it.
6. Write `hash_password(password, salt)` using PBKDF2.
7. Write `verify_password(password, salt, expected_hash)` using PBKDF2.
8. Demonstrate `crypto.constant_time_eq` vs regular == for timing safety.

8 Chapter 26: Networking Fundamentals

Zen's networking stack provides sockets, DNS, HTTP, TLS, and raw packet crafting. This chapter covers the fundamentals: IP addressing, TCP vs UDP, ports, the OSI model mapping, and how Zen's networking modules fit together.

8.1 26.1 Networking module map

Zen's networking is split across several modules, each at a different layer:

Module	Layer	Purpose
`socket`	Transport (L4)	TCP/UDP sockets, `connect`, `bind`, `listen`, `accept`, `send`, `recv`
`net`	Transport (L4)	Higher-level TCP/UDP helpers (`tcp_connect`, `tcp_send`, etc.)
`net.dns`	Application (L7)	DNS resolution (`resolve`, `gethostbyname`)
`requests`	Application (L7)	HTTP client (GET/POST/etc., sessions, TLS)
`http`	Application (L7)	Low-level HTTP client
`http.server`	Application (L7)	Static file HTTP server
`tls`	Security (L4.5)	TLS connections with certificate pinning
`raw` / `net_raw`	Network (L3)	Raw IP sockets, packet crafting (`pack_ip`, `pack_tcp`)
`sys`	System	File descriptors, pipes, syscalls

8.2 26.2 IP addressing

Zen works with IPv4 addresses (dotted-decimal strings like `"192.168.1.1"`). IPv6 support depends on the platform.

```
import socket
import dns

// Resolve a hostname to IP addresses
let addrs = dns.resolve("example.com")
for addr in addrs {
    print(addr)  # "93.184.216.34" or "2606:2800:220:1:248:1893:25c8:1946"
}

// localhost
print(dns.resolve("localhost"))  # ["127.0.0.1", "::1"]
```

8.2.1 Private IP ranges (RFC 1918)

Range	Purpose
`10.0.0.0` – `10.255.255.255`	Private (class A)
`172.16.0.0` – `172.31.255.255`	Private (class B)
`192.168.0.0` – `192.168.255.255`	Private (class C)
`127.0.0.0` – `127.255.255.255`	Loopback (localhost)
`169.254.0.0` – `169.254.255.255`	Link-local (APIPA)

```
import dns

// Localhost always resolves
print(dns.resolve("127.0.0.1"))   # ["127.0.0.1"]
print(dns.resolve("localhost"))   # ["127.0.0.1", ...]
```

8.3 26.3 Ports

Ports are 16-bit integers (0–65535). Ports 0–1023 are “well-known” and typically require root privileges to bind.

Port	Service
20	FTP data
21	FTP control
22	SSH
23	Telnet
25	SMTP
53	DNS
67/68	DHCP
80	HTTP
110	POP3
123	NTP
143	IMAP
161/162	SNMP
389	LDAP
443	HTTPS
465	SMTPS
587	SMTP submission
636	LDAPS
993	IMAPS
995	POP3S
1433	MSSQL
3306	MySQL
3389	RDP
5432	PostgreSQL
5900	VNC
6379	Redis
27017	MongoDB

Port	Service
8080	HTTP alternate

```
import socket

// Bind to port 80 typically requires root
let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 80)   # may fail without root
socket.bind(server, "0.0.0.0", 8080)  # usually OK
```

8.4 26.4 TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Reliable (retransmission, ACK)	Unreliable (no guarantee)
Ordering	Ordered	Unordered
Flow control	Yes	No
Congestion control	Yes	No
Header overhead	20+ bytes	8 bytes
Use case	Web, file transfer, email	DNS, streaming, VoIP, games

```
import socket

// TCP socket (stream)
let tcp = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(tcp, "example.com", 80)
socket.send(tcp, "GET / HTTP/1.0\r\n\r\n")
let resp = socket.recv(tcp, 4096)
socket.close(tcp)

// UDP socket (datagram)
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.send_to(udp, "query", "8.8.8.8", 53)
let [data, addr] = socket.recv_from(udp, 4096)
socket.close(udp)
```

8.5 26.5 The OSI model and Zen

OSI Layer	Name	Zen module	Example
7	Application	`requests`, `http`, `http.server`, `dns`	HTTP GET, DNS resolve
6	Presentation	`base64`, `binascii`, `crypto`	Encoding, encryption
5	Session	`requests.Session`	HTTP sessions
4	Transport	`socket`, `net`, `tls`	TCP, UDP, TLS
3	Network	`raw`, `net_raw`	IP packets, ICMP
2	Data Link	—	(not exposed)
1	Physical	—	(not exposed)

Zen focuses on layers 3–7. Layers 1–2 (physical, data link) are handled by the OS/kernel.

8.6 26.6 Sockets: file descriptors

Zen sockets use integer file descriptors (handles) into an internal resource table:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
print(sock)   # 3 (an integer handle)
socket.connect(sock, "example.com", 80)
socket.close(sock)   # release the handle
```

Always close sockets when done to avoid resource leaks:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
try {
    socket.connect(sock, "example.com", 80)
    socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
    let resp = socket.recv(sock, 4096)
    print(resp)
} finally {
    socket.close(sock)   # always close
}
```

8.7 26.6 Address families

Constant	Meaning
`socket.AF_INET`	IPv4
`socket.AF_INET6`	IPv6 (platform-dependent)
`socket.AF_UNIX`	Unix domain sockets (platform-dependent)

8.7.1 Socket types

Constant	Meaning
`socket.SOCK_STREAM`	TCP (reliable stream)
`socket.SOCK_DGRAM`	UDP (unreliable datagram)
`socket.SOCK_RAW`	Raw IP (requires root)

```
import socket

let tcp = socket.create(socket.AF_INET, socket.SOCK_STREAM)
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
let raw = socket.create(socket.AF_INET, socket.SOCK_RAW)  # may need root
```

8.8 26.6 Timeouts

Socket operations can block indefinitely. Always set timeouts:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)   # 5 second timeout

try {
    socket.connect(sock, "slow.example.com", 80)
    socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
    let resp = socket.recv(sock, 4096)
    print(resp)
} catch as e {
    print(e.msg)  # timeout or connection error
} finally {
    socket.close(sock)
}
```

8.9 26.6 Byte order

Network byte order is big-endian. Zen's socket functions handle this internally. When crafting raw packets with `net_raw`, you must manually use big-endian:

```
import net_raw

// pack_ip and pack_tcp handle byte order internally
let pkt = net_raw.pack_ip("1.2.3.4", "5.6.7.8", 6)
```

8.10 26.7 Quick reference

```
import socket
import dns
import net_raw

// Resolve
let addrs = dns.resolve("example.com")

// TCP client
let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)
socket.connect(sock, addrs[0], 80)
socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let resp = socket.recv(sock, 4096)
socket.close(sock)

// UDP
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.send_to(udp, "ping", "127.0.0.1", 9999)
let [data, addr] = socket.recv_from(udp, 4096)
socket.close(udp)

// Raw IP packet
let ip_pkt = net_raw.pack_ip("1.2.3.4", "5.6.7.8", 6)
let tcp_pkt = net_raw.pack_tcp(12345, 80, "SYN")
```

8.11 26.8 Exercises

1. Write a function `resolve_all(host)` that prints all IP addresses for a hostname.
2. Write a TCP echo client that sends a line and prints the response.
3. Write a UDP client that sends a datagram and waits for a reply.
4. Write a port scanner for ports 1–1000 on localhost with a 100ms timeout per port.
5. Create a raw IP packet with `net_raw.pack_ip` and inspect the first 20 bytes.
6. Write a function `is_port_open(host, port, timeout)` that returns true if a TCP connect succeeds.
7. Demonstrate that `socket.close` releases the file descriptor by opening and closing multiple sockets.

9 Chapter 27: socket Module

The socket module provides low-level TCP/UDP/raw sockets. It's Tier 1 of Zen's networking stack — thin wrappers over the OS socket API with integer file descriptors (handles).

9.1 27.1 Socket creation

```
import socket

// TCP socket (IPv4)
let tcp = socket.create(socket.AF_INET, socket.SOCK_STREAM)

// UDP socket (IPv4)
let udp = socket.create(socket.AF_INET, socket.SOCK_DGRAM)

// Raw socket (requires root on most OSes)
let raw = socket.create(socket.AF_INET, socket.SOCK_RAW)
```

9.1.1 Address families and socket types

Constant	Meaning
socket.AF_INET	IPv4
socket.AF_INET6	IPv6 (platform-dependent)
socket.AF_UNIX	Unix domain sockets (platform-dependent)
socket.SOCK_STREAM	TCP (reliable stream)
socket.SOCK_DGRAM	UDP (unreliable datagram)
socket.SOCK_RAW	Raw IP (requires root)

9.1.2 Common combinations

Family	Type	Protocol	Use
AF_INET	SOCK_STREAM	TCP	Web, file transfer, email
AF_INET	SOCK_DGRAM	UDP	DNS, streaming, games
AF_INET	SOCK_RAW	IP	Packet crafting, ping
AF_INET6	SOCK_STREAM	TCP over IPv6	IPv6 services
AF_UNIX	SOCK_STREAM	Local	IPC on same machine

9.2 27.2 Socket handle operations

Function	Description
socket.create(domain, type)	Create socket, returns handle (int)

Function	Description
`socket.connect(sock, host, port)`	Connect TCP socket to remote
`socket.bind(sock, host, port)`	Bind socket to local address
`socket.listen(sock, backlog)`	Listen for incoming connections
`socket.accept(sock)`	Accept incoming connection, returns new socket handle
`socket.send(sock, data)`	Send data on connected socket
`socket.recv(sock, n)`	Receive up to n bytes (returns bytes)
`socket.recv_text(sock, n)`	Receive text
`socket.recv_from(sock, n)`	Receive from address (UDP), returns `{data, addr}`
`socket.send_to(sock, data, host, port)`	Send to address (UDP)
`socket.settimeout(sock, ms)`	Set timeout in milliseconds
`socket.close(sock)`	Close socket and release handle
`socket.fileno(sock)`	Get raw file descriptor

9.3 27.3 TCP client

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)   // 5 second timeout

socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")

let response = ""
while true {
    let chunk = socket.recv(sock, 4096)
    if len(chunk) == 0 {
        break
    }
    response = response + chunk
}

print(response)
socket.close(sock)
```

9.3.1 recv returns bytes

`socket.recv` returns a byte list (list of ints 0–255), not a string. Use `socket.recv_text` for text, or convert:

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\n\r\n")

let bytes = socket.recv(sock, 1024)
// bytes is a list like [72, 84, 84, 80, ...]

let text = socket.recv_text(sock, 1024)
// text is a string

socket.close(sock)
```

To convert bytes to string manually:

```
import socket

let bs = socket.recv(sock, 1024)
let s = ""
for b in bs {
    s = s + chr(b)
}
print(s)
```

9.4 27.4 TCP server

```
import socket

let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 8080)
socket.listen(server, 5)

print("Listening on 0.0.0.0:8080")

let client = socket.accept(server)
let req = socket.recv_text(client, 4096)
print("Request: " + req)

socket.send(client, "HTTP/1.0 200 OK\r\nContent-Type: text/plain\r\n\r\nHello, Zen!")
socket.close(client)
socket.close(server)
```

9.4.1 Accept loop

```
import socket

let server = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.bind(server, "0.0.0.0", 8080)
socket.listen(server, 5)

while true {
    let client = socket.accept(server)
    let req = socket.recv_text(client, 4096)
    socket.send(client, "HTTP/1.0 200 OK\r\n\r\npong")
    socket.close(client)
}
```

9.5 27.5 UDP

UDP is connectionless — no `connect`, `listen`, or `accept`. Use `send_to` and `recv_from`.

9.5.1 UDP client

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.settimeout(sock, 5000)

socket.send_to(sock, "ping", "127.0.0.1", 9999)

let [data, addr] = socket.recv_from(sock, 4096)
print("Response from " + addr.addr + ": " + data)

socket.close(sock)
```

`recv_from` returns a dict `{data: <bytes>, addr: <ip_string>}`.

9.5.2 UDP server

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_DGRAM)
socket.bind(sock, "0.0.0.0", 9999)

while true {
    let [data, addr] = socket.recv_from(sock, 4096)
    print("From " + addr.addr + ": " + data)
    socket.send_to(sock, "pong", addr.addr, 9999)
}
```

9.6 27.6 Socket options

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)

// Set timeout
socket.settimeout(sock, 5000)  // 5 seconds

// Get file descriptor (for raw operations)
let fd = socket.fileno(sock)
print(fd)  // integer

socket.close(sock)
```

9.7 27.7 Common patterns

9.7.1 Read until connection closes

```
import socket

func read_all(sock) {
    let result = ""
    while true {
        let chunk = socket.recv_text(sock, 4096)
        if len(chunk) == 0 {
            break
        }
        result = result + chunk
    }
    return result
}

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let body = read_all(sock)
print(body)
socket.close(sock)
```

9.7.2 Read exactly n bytes

```
import socket

func recv_exact(sock, n) {
    let buf = []
```

```
    while len(buf) < n {
        let chunk = socket.recv(sock, n - len(buf))
        if len(chunk) == 0 {
            break
        }
        for b in chunk {
            buf.push(b)
        }
    }
    return buf
}
```

9.7.3 Port scanner

```
import socket

func scan_port(host, port, timeout) {
    try {
        let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
        socket.settimeout(sock, timeout)
        socket.connect(sock, host, port)
        socket.close(sock)
        return true
    } catch {
        return false
    }
}

for port in 1..1024 {
    if scan_port("127.0.0.1", port, 100) {
        print("Port " + str(port) + " is open")
    }
}
```

9.7.4 HTTP client (manual, no requests module)

```
import socket

func http_get(host, path) {
    let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
    socket.connect(sock, host, 80)

    let request = "GET " + path + " HTTP/1.0\r\nHost: " + host + "\r\nConnection: close
    socket.send(sock, request)

    let response = ""
```

```
    while true {
        let chunk = socket.recv_text(sock, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }

    socket.close(sock)
    return response
}

print(http_get("example.com", "/"))
```

9.8 27.8 Common pitfalls

- **Always close sockets.** Unclosed sockets leak file descriptors.
- **Set timeouts.** Without timeouts, `connect`/`recv` can block forever.
- **TCP is a stream.** `recv` may return fewer bytes than requested — loop until you have all data.
- **`recv` returns bytes**, not strings. Use `recv_text` or convert manually.
- **UDP is connectionless.** Use `send_to`/`recv_from`, not `connect`/`send`/`recv`.
- **`recv_from` returns `{data, addr}`**, not just data.
- **Raw sockets (`SOCK_RAW`) require root** on most operating systems.

9.9 27.8 Quick reference

```
import socket

let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.settimeout(sock, 5000)
socket.connect(sock, "example.com", 80)
socket.send(sock, "GET / HTTP/1.0\r\n\r\n")
let data = socket.recv(sock, 4096)
let text = socket.recv_text(sock, 4096)
socket.close(sock)

// UDP
socket.send_to(sock, "data", "127.0.0.1", 9999)
let [data, addr] = socket.recv_from(sock, 4096)

// Server
socket.bind(sock, "0.0.0.0", 8080)
socket.listen(sock, 5)
let client = socket.accept(sock)
```

9.10 26.9 Exercises

1. Write a TCP echo client that sends a line and prints the response.
2. Write a UDP client that sends a datagram and waits for a reply.
3. Write a port scanner for ports 1–1000 on localhost.
4. Write a simple HTTP/0.9 client that fetches the root page.
5. Write a TCP echo server that handles one connection at a time.
6. Write a UDP server that echoes back datagrams.
7. Combine client and server in one script (server in a thread, client in main).

10 Chapter 28: HTTP Protocol

Zen's `http` module provides a low-level HTTP/1.1 client. For a higher-level experience with sessions, cookies, and JSON, use the `requests` module (Chapter 29).

10.1 28.1 HTTP basics

HTTP (Hypertext Transfer Protocol) is the foundation of the web. A client sends a request; a server responds.

10.1.1 Request anatomy

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Zen/3.0
Accept: text/html
```

- **Method**: GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH
- **Path**: `/index.html`
- **Version**: `HTTP/1.1`
- **Headers**: `Host`, `User-Agent`, `Accept`, etc.
- **Body**: empty for GET, present for POST/PUT

10.1.2 Response anatomy

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 137

<html><body>Hello</body></html>
```

- **Status line**: `HTTP/1.1 200 OK`
- **Headers**: `Content-Type`, `Content-Length`, etc.
- **Body**: the actual content (may be empty)

10.2 28.2 The `http` module

Zen's `http` module provides low-level HTTP/1.1 client functions:

```
import http

let resp = http.get("https://example.com")
print(resp.status_code)   # 200
print(resp.body)          # response body
print(resp.headers)       # headers dict
```

10.2.1 http functions

Function	Description
`http.get(url, headers?)`	GET request
`http.post(url, body, headers?)`	POST request
`http.put(url, body, headers?)`	PUT request
`http.delete(url, headers?)`	DELETE request
`http.head(url, headers?)`	HEAD request
`http.options(url, headers?)`	OPTIONS request
`http.patch(url, body, headers?)`	PATCH request

10.2.2 Response object

Field	Description
`status_code`	HTTP status code (int)
`reason`	Reason phrase (string)
`headers`	Response headers (dict)
`body`	Response body (string)
`ok`	True if 200–299

```
import http

let resp = http.get("https://example.com")
print(resp.status_code)   # 200
print(resp.reason)        # OK
print(resp.body)          # HTML body
print(resp.headers)       # headers dict
print(resp.ok)            # true

if resp.ok {
    print("Success!")
} else {
    print("Error: " + str(resp.status_code) + " " + resp.reason)
}
```

10.3 28.3 Making requests with headers

```
import http

let resp = http.get("https://api.example.com/user/1", {
    "User-Agent": "Zen/3.0",
    "Accept": "application/json",
    "Authorization": "Bearer token123"
})
```

```
print(resp.status_code)
print(resp.body)
```

10.4 28.4 POST with body

```
import http

let body = "name=Zen&version=3"
let resp = http.post("https://example.com/api", body, {
    "Content-Type": "application/x-www-form-urlencoded",
    "Content-Length": str(len(body))
})

print(resp.status_code)
print(resp.body)
```

10.5 28.5 Status codes

Code	Name	Meaning
200	OK	Success
201	Created	Resource created
204	No Content	Success, no body
301	Moved Permanently	Redirect
302	Found	Temporary redirect
304	Not Modified	Cache hit
400	Bad Request	Client error
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource not found
500	Internal Server Error	Server error
502	Bad Gateway	Upstream error
503	Service Unavailable	Server overloaded

```
import http

let resp = http.get("https://example.com/nonexistent")
print(resp.status_code)   # 404
print(resp.ok)            # false
```

10.6 28.6 HTTP vs HTTPS

Zen's `http` module supports both `http://` and `https://` URLs. HTTPS uses TLS under the hood (certificate validation via the platform root store).

```
import http

// HTTP (port 80)
let r1 = http.get("http://example.com")

// HTTPS (port 443, TLS)
let r2 = http.get("https://example.com")
```

For certificate pinning, use the `tls` module directly (Chapter 25) or `requests` with verify options (Chapter 29).

10.7 28.7 Common patterns

10.7.1 Simple GET

```
import http

let resp = http.get("https://example.com")
if resp.ok {
    print(resp.body)
} else {
    print("HTTP " + str(resp.status_code))
}
```

10.7.2 POST JSON

```
import http
import json

let body = json.dumps({"name": "Zen", "version": 3})
let resp = http.post("https://example.com/api", body, {
    "Content-Type": "application/json",
    "Content-Length": str(len(body))
})

print(resp.status_code)
print(resp.body)
```

10.7.3 Check if resource exists (HEAD)

```
import http

let resp = http.head("https://example.com/image.png")
if resp.ok {
    print("Resource exists, size: " + resp.headers["Content-Length"])
} else {
```

```
    print("Resource not found")
}
```

10.8 28.8 Common pitfalls

- **`http` is low-level.** No automatic redirect following, no session cookies, no connection pooling. Use `requests` for those features.
- **Always check `resp.ok`.** A non-200 response still returns a response object — it doesn't raise automatically.
- **HTTPS requires TLS support.** If Zen is built without TLS, `https://` URLs will fail.
- **`Content-Length` must match** the actual body length when sending POST/PUT.
- **Redirects are not followed** automatically by `http`. Use `requests` for redirect following.

10.9 28.9 Quick reference

```
import http

let resp = http.get("https://example.com", {
    "User-Agent": "Zen/3.0"
})

resp.status_code   # 200
resp.reason        # "OK"
resp.headers       # headers dict
resp.body          # response body
resp.ok            # true if 200-299

let resp2 = http.post("https://example.com/api", "data", {
    "Content-Type": "application/json"
})
```

10.10 28.10 Exercises

1. Write a function `fetch_title(url)` that fetches a page and extracts the `<title>` tag.
2. Write a script that checks if a list of URLs return 200 or 404.
3. Write a `head_check(url)` that returns the `Content-Length` header without downloading the body.
4. Write a POST script that sends form data (`application/x-www-form-urlencoded`).
5. Compare `http.get` vs `requests.get` — note the differences in convenience features.

11 Chapter 31: HTTPS & SSL/TLS Certificates

Zen's `tls` module provides encrypted TCP connections using rustls, with optional certificate pinning. This chapter covers TLS connections, certificate pinning, and secure HTTPS practices.

11.1 31.1 The `tls` module

```
import tls

fd = tls.connect("example.com", 443)
tls.write(fd, "GET / HTTP/1.0\r\nHost: example.com\r\n\r\n")
let resp = tls.read(fd, 4096)
print(resp)
tls.close(fd)
```

11.1.1 TLS functions

Function	Description
`tls.connect(host, port)`	Plain TLS connection, returns fd
`tls.connect_pinned(host, port, pins)`	TLS with certificate pinning
`tls.write(fd, data)`	Send data, returns nil
`tls.read(fd, n)`	Read up to n bytes, returns string
`tls.close(fd)`	Close connection, returns nil

```
import tls

let fd = tls.connect("example.com", 443)
tls.write(fd, "GET / HTTP/1.0\r\nHost: example.com\r\nConnection: close\r\n\r\n")
let resp = tls.read(fd, 4096)
print(resp)
tls.close(fd)
```

11.2 31.2 Certificate pinning

Certificate pinning ensures you only connect to servers with a specific certificate, preventing man-in-the-middle attacks even if the CA is compromised.

```
import tls

let pins = ["sha256:AAAA...BBBB..."]  // SHA-256 of server's SPKI
let fd = tls.connect_pinned("example.com", 443, pins)
tls.write(fd, "GET / HTTP/1.0\r\n\r\n")
let resp = tls.read(fd, 4096)
tls.close(fd)
```

11.2.1 How pinning works

1. `tls.connect_pinned` connects to the server.
2. During the TLS handshake, the server presents its certificate.
3. Zen computes the SHA-256 hash of the certificate's SPKI (Subject Public Key Info).
4. If the hash matches one of the pins, the connection proceeds.
5. If no pin matches, the handshake fails.

11.2.2 Getting a certificate's SPKI hash

```
// You can compute this externally with openssl:
// openssl s_client -connect example.com:443 -showcerts
// Then hash the SPKI of the leaf certificate.

// Example pin format:
let pins = ["sha256:a1b2c3d4e5f6..."]
```

11.3 31.3 Plain TLS vs pinned TLS

Feature	`tls.connect`	`tls.connect_pinned`
Verification	Platform root store (webpki-roots)	Pin SPKI hash match
Trust model	CA hierarchy	Explicit pins
Fallback	Accepts any CA-signed cert	Only pinned certs
Use case	General browsing	Known services (APIs, internal)

```
import tls

// General browsing - trusts any CA-signed cert
let fd1 = tls.connect("example.com", 443)

// Known service - only accept specific cert
let pins = ["sha256:expected-spki-hash"]
let fd2 = tls.connect_pinned("api.internal.com", 443, pins)
```

11.4 31.4 Complete HTTPS client example

```
import tls

func https_get(host, path) {
    let fd = tls.connect(host, 443)

    let request = "GET " + path + " HTTP/1.0\r\n" +
```

```
                  "Host: " + host + "\r\n" +
                  "Connection: close\r\n\r\n"

    tls.write(fd, request)

    let response = ""
    while true {
        let chunk = tls.read(fd, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }

    tls.close(fd)
    return response
}

print(https_get("example.com", "/"))
```

11.4.1 HTTPS with pinning

```
import tls

func https_get_pinned(host, path, pins) {
    let fd = tls.connect_pinned(host, 443, pins)

    let request = "GET " + path + " HTTP/1.0\r\n" +
                  "Host: " + host + "\r\n" +
                  "Connection: close\r\n\r\n"

    tls.write(fd, request)

    let response = ""
    while true {
        let chunk = tls.read(fd, 4096)
        if len(chunk) == 0 {
            break
        }
        response = response + chunk
    }

    tls.close(fd)
    return response
}
```

```
let pins = ["sha256:AAAA...BBBB..."]
print(https_get_pinned("example.com", "/", pins))
```

11.5 31.5 Error handling

```
import tls

try {
    let fd = tls.connect("nonexistent.example.invalid", 443)
    tls.write(fd, "GET / HTTP/1.0\r\n\r\n")
    let resp = tls.read(fd, 4096)
    tls.close(fd)
} catch as e {
    print("TLS error: " + e.msg)
    // Could be: DNS failure, connection refused, TLS handshake failure,
    // certificate verification failure, pin mismatch
}

// Pin mismatch example
try {
    let pins = ["sha256:wrong-hash"]
    let fd = tls.connect_pinned("example.com", 443, pins)
} catch as e {
    print(e.msg)  // certificate pin mismatch
}
```

11.6 31.6 Quick reference

```
import tls

let fd = tls.connect("example.com", 443)
tls.write(fd, "GET / HTTP/1.0\r\n\r\n")
let resp = tls.read(fd, 4096)
tls.close(fd)

// With pinning
let pins = ["sha256:AAAA..."]
let fd2 = tls.connect_pinned("example.com", 443, pins)
tls.close(fd2)
```

11.7 31.7 Exercises

1. Write an HTTPS client that fetches a page and prints the status line.
2. Get a certificate's SPKI hash using openssl, then use `tls.connect_pinned` to connect only to that cert.

3. Write a function `https_get_pinned(host, path, pins)` that returns the response body.
4. Demonstrate that `tls.connect_pinned` fails with a wrong pin.
5. Write a script that connects to an HTTPS API and parses the JSON response.

12 Chapter 35: Threading Fundamentals

Zen supports OS threads via the `thread` module. Threads run concurrently (subject to the OS scheduler) and communicate through `shared` atomic values and `channel` message passing.

12.1 35.1 The `thread` module

```
import thread

func worker(name) {
    print("Worker " + name + " starting")
    return "done: " + name
}

let t = thread.spawn(worker("A"))
let result = thread.join(t)
print(result)  # done: A

print(thread.current())  # current thread info (nil or handle)
```

12.1.1 Thread functions

Function	Description	Example
`thread.spawn(func)`	Start `func` in a new thread	`thread.spawn(worker)`
`thread.join(thread)`	Wait for thread, get return value	`thread.join(t)`
`thread.current()`	Current thread info	`thread.current()`

```
import thread

func count(n) {
    let total = 0
    for i in 1..n {
        total = total + i
    }
    return total
}

let t = thread.spawn(count(100))
let result = thread.join(t)
print(result)  # 5050
```

12.2 35.2 Thread lifecycle

```
import thread

func slow() {
    // simulate work
    return "finished"
}

let t = thread.spawn(slow)

// Do other work in main thread...

let result = thread.join(t)  // blocks until slow() returns
print(result)
```

`thread.join` blocks the calling thread until the target thread completes, then returns the target's return value. If the target raised an error, `join` propagates it.

```
import thread

func failing() {
    throw "boom"
}

let t = thread.spawn(failing)
try {
    thread.join(t)
} catch as e {
    print(e.msg)  # boom
}
```

12.3 35.3 Thread-local data

Variables created inside a thread function are local to that thread:

```
import thread

func worker() {
    let local = "only in this thread"
    print(local)
}

let t = thread.spawn(worker)
thread.join(t)
// local is not accessible here
```

To share data between threads, use `shared` (atomic values) or `channel` (message passing).

12.4 35.3 Multiple threads

```
import thread

func worker(id) {
    print("Worker " + str(id) + " started")
    return "worker " + str(id) + " done"
}

let threads = []
for i in 1..5 {
    let t = thread.spawn(worker(i))
    threads.push(t)
}

for t in threads {
    let result = thread.join(t)
    print(result)
}
print("All workers done")
```

12.5 35.4 Common pitfalls

- **`thread.join` blocks.** Join all threads before your program exits, or their work may be cut short.
- **Exceptions propagate.** If a thread function raises, `thread.join` re-raises it in the joining thread.
- **No shared state by default.** Each thread has its own stack and locals. Use `shared` or `channel` to communicate.
- **Thread count.** Don't spawn more threads than necessary; OS threads are relatively heavyweight.

12.6 35.5 Quick reference

```
import thread

let t = thread.spawn(func() { return "result" })
let result = thread.join(t)
let cur = thread.current()
```

12.7 35.6 Exercises

1. Write a program that spawns 5 threads, each printing its ID, and joins them all.
2. Write `parallel_sum(nums)` that splits a list into chunks, sums each chunk in a thread, and combines results.
3. Write a thread that sleeps for 1 second then prints "done".
4. Demonstrate that `thread.join` propagates errors from the child thread.

5. Write a function `run_concurrent(funcs)` that runs a list of zero-arg functions in threads and returns their results in order.

13 Chapter 36: Channels

Channels send values between threads. A channel is created, values are sent into it, and received from it on the other end. Channels are unbuffered — `send` blocks until a `recv` is waiting, and vice versa.

13.1 36.1 The `channel` module

```
import channel

let ch = channel.make()

channel.send(ch, "hello")
channel.send(ch, 42)
channel.send(ch, true)

print(channel.recv(ch))  # hello
print(channel.recv(ch))  # 42
print(channel.recv(ch))  # true
```

13.1.1 Channel functions

Function	Description	Example
`channel.make()`	Create a new channel	`channel.make()`
`channel.send(ch, value)`	Send a value (blocks until received)	`channel.send(ch, 42)`
`channel.recv(ch)`	Receive a value (blocks until sent)	`channel.recv(ch)`

```
import channel

let ch = channel.make()

channel.send(ch, "first")
channel.send(ch, "second")
channel.send(ch, "third")

print(channel.recv(ch))  # first
print(channel.recv(ch))  # second
print(channel.recv(ch))  # third
```

13.2 36.2 Blocking semantics

Channels are unbuffered and synchronous:

- `send(ch, v)` blocks until some thread calls `recv(ch)`.

- `recv(ch)` blocks until some thread calls `send(ch, v)`.
- The value is handed directly from sender to receiver.

```
import channel
import thread

let ch = channel.make()

// Sender thread
thread.spawn(fn() {
    channel.send(ch, "ping")
    print("sent")
})

// Receiver (main thread)
let msg = channel.recv(ch)
print("received: " + msg)  // received: ping
```

13.3 36.3 Producer-consumer

```
import channel
import thread

let ch = channel.make()

func producer() {
    for i in 1..5 {
        channel.send(ch, i)
        print("produced: " + str(i))
    }
    channel.send(ch, nil)  // sentinel: done
}

func consumer() {
    while true {
        let v = channel.recv(ch)
        if v == nil {
            break
        }
        print("consumed: " + str(v))
    }
}

let t1 = thread.spawn(producer)
let t2 = thread.spawn(consumer)
thread.join(t1)
thread.join(t2)
```

```
print("all done")
```

Output:

```
produced: 1
consumed: 1
produced: 2
consumed: 2
...
produced: 5
consumed: 5
all done
```

13.4 36.4 Multiple producers and consumers

```
import channel
import thread

let ch = channel.make()

func producer(id, count) {
    for i in 1..count {
        channel.send(ch, #(id, i))
    }
}

func consumer(id) {
    while true {
        let v = channel.recv(ch)
        if v == nil {
            break
        }
        print("consumer " + str(id) + " got: " + str(v[1]))
    }
}

// Start 2 producers
thread.spawn(producer(1, 3))
thread.spawn(producer(2, 3))

// Send 2 sentinels (one per consumer)
channel.send(ch, nil)
channel.send(ch, nil)

// Start 2 consumers
thread.spawn(consumer(1))
thread.spawn(consumer(2))
```

13.5 36.5 Channels and shared values

Use `shared` for counters, `channel` for message passing:

```
import channel
import shared
import thread

let ch = channel.make()
let counter = shared.create(0)

func worker(id) {
    for i in 1..10 {
        shared.increment(counter, 1)
        channel.send(ch, #(id, i))
    }
}

let t1 = thread.spawn(worker(1))
let t2 = thread.spawn(worker(2))

// Collect results
for i in 1..20 {
    let v = channel.recv(ch)
    print("got: " + str(v))
}

thread.join(t1)
thread.join(t2)
print("counter: " + str(shared.get(counter)))  # 20
```

13.6 36.6 Common pitfalls

- **`recv` blocks forever** if no value is ever sent. Always ensure a sender exists, or use a sentinel value (like `nil`) to signal completion.
- **`send` blocks forever** if no receiver is waiting.
- **Channels are unbuffered.** There's no queue — sender and receiver must rendezvous.
- **Forgetting to join threads.** The program may exit before threads finish sending.

13.7 36.7 Quick reference

```
import channel

let ch = channel.make()
channel.send(ch, "msg")
let v = channel.recv(ch)
```

```
// Producer-consumer pattern
channel.send(ch, nil)  // sentinel to signal completion
```

13.8 36.8 Exercises

1. Write a program with 1 producer and 1 consumer using a channel.
2. Write a program with 2 producers and 1 consumer.
3. Write `parallel_map(func, items)` that runs `func` on each item in a thread and sends results via a channel.
4. Write a pipeline: stage1 generates numbers, stage2 squares them, stage3 prints them. Connect stages with channels.
5. Write a program where multiple workers pull jobs from a channel channel and send results to another channel.

14 Chapter 42: pymport - Python Interoperability

`pymport` imports Python modules into Zen, letting you call Python functions, access Python objects, and use the entire Python ecosystem from Zen. This chapter covers setup, syntax, `zen py-sync`, environment management, limits, examples, and troubleshooting.

14.1 42.1 Why `pymport` (and not just `import`)?

Zen's `import` loads Zen modules (`.z` files or native Rust modules). `pymport` loads **CPython** modules — the entire Python ecosystem (PyPI, numpy, pandas, requests, beautifulsoup4, etc.).

```
import json          // Zen's json module (Zen or native Rust)
pymport json         // CPython's json module
```

14.2 42.2 Syntax and Binding

`pymport` calls are **positional-only** — no keyword arguments.

```
pymport json
pymport os
pymport requests
pymport bs4            // beautifulsoup4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins
```

14.2.1 Calling functions

```
pymport json
let data = json.loads('{"name": "Zen", "version": 3}')
print(data["name"])     # Zen

pymport os
print(os.getcwd())
print(os.path.join("a", "b"))

pymport datetime
let now = datetime.datetime.now()
print(now.year)
print(now.isoformat())
```

14.2.2 Attribute access and method calls

Foreign objects (e.g. `datetime.datetime`) are opaque handles — attributes and methods work:

```
pymport datetime
let now = datetime.datetime.now()
print(now.year)           // attribute access
print(now.isoformat())    // method call
let delta = datetime.timedelta(days=7)
let later = now + delta   // operator overloading (if supported)
```

Scalar results marshal back to Zen values (int, float, string, bool, list, dict).

14.3 42.3 Building Zen with Python Support

`pymport` requires Zen built with the Python feature:

```
cargo build --release --features python
```

This requires `libpython` at build and run time. Without it, `pymport` reports:

```
pymport needs a linked Python bridge (this build has none). Rebuild with --
features python
```

14.4 42.4 Environment Setup (venv + PYTHONPATH)

You can point CPython to any Python environment via `PYTHONPATH`.

```
cd book-v2/tests
./setup.sh          # builds zen --features python, creates .venv, installs requirement

export PYTHONPATH="$(book-v2/tests/.venv/bin/python -c 'import sysconfig; print(sysconf
export ZEN_NO_PY_BOOTSTRAP=1
./target/release/zen run script.z
```

`ZEN_NO_PY_BOOTSTRAP=1` tells Zen to use that environment as-is instead of creating `.zen_venv`.

14.4.1 Create your own venv

```
python3 -m venv .venv
.venv/bin/pip install beautifulsoup4 mechanicalsoup selenium lxml requests
export PYTHONPATH="$(.venv/bin/python -c 'import sysconfig; print(sysconfig.get_paths()
```

14.4.2 Check installed packages

```
book-v2/tests/.venv/bin/pip list
```

14.5 42.5 Dependency Sync with `zen py-sync` and `requirements.txt`

Zen can scan your `.z` files for `pymport` statements and ensure a venv with dependencies.

14.5.1 Create `requirements.txt`

```
beautifulsoup4
mechanicalsoup
selenium
requests
lxml
```

> A module name is not always the pip name (e.g., `bs4` comes from `beautifulsoup4`). The scanner reports what was found; Zen does **not** guess package names.

14.5.2 Run sync

```
./target/release/zen py-sync
```

14.5.3 Behavior

1. **Scan**: recursively parses entry + transitive `.z` imports to find all `pymport` statements.
2. **Plan**: determines state (`Clean`, `NoRequirements`, `MissingVenv`, `MissingPackages`).
3. **Ensure**: creates `.zen_venv` in project root, runs `pip install -r requirements.txt` in the foreground, writes a stamp (requirements length + hash).
4. **Point**: `zen run` prepends `.zen_venv/lib/python*/site-packages` to `PYTHONPATH`.

14.5.4 Control flags

Flag	Meaning
`ZEN_NO_PY_BOOTSTRAP=1`	Skip bootstrap entirely (for tests/hermetic runs).
(none)	If `requirements.txt` is missing and `pymport` is used, Zen reports an error.

14.5.5 Example end-to-end

```
cat > requirements.txt << 'EOF'
paramiko
EOF
./target/release/zen py-sync
./target/release/zen run my_script.z  # uses .zen_venv
```

14.6 42.6 Using Python from Zen

14.6.1 Importing modules

```
pymport json        // stdlib
pymport os          // stdlib
pymport requests    // third-party (if installed)
pymport bs4         // beautifulsoup4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins
```

14.6.2 Calling functions and accessing attributes

Foreign calls are **positional-only** in v1. Use `getattr`-style access via dot notation; `call_attr` fuses resolve+dispatch for methods.

```
pymport json
let data = json.loads('{"a": 1}')
print(data["a"])

pymport os
print(os.getcwd())
print(os.path.join("a", "b"))

pymport datetime
let now = datetime.datetime.now()
print(now.year)
print(now.isoformat())
```

14.6.3 Working with Python objects and binary data

Foreign objects are opaque handles:

```
pymport datetime
pymport builtins

let epoch = datetime.datetime.fromtimestamp(0)
print(epoch.year)
print(epoch.isoformat())
print(epoch.date().isoformat())

let later = datetime.datetime(2030, 1, 2, 3, 4, 5)
print(later.strftime("%Y-%m-%d %H:%M:%S"))

print(builtins.len([10, 20, 30]))
print(builtins.sum([10, 20, 30]))
print(builtins.sorted([3, 1, 2]))
```

14.6.4 Binary data

Python `bytes` are foreign handles. Write them through Python's own file object:

```
import fs
pymport mechanicalsoup
pymport builtins

let f = builtins.open("out.bin", "wb")
f.write(some_bytes)
f.close()
```

14.7 42.7 Error handling

Python exceptions become catchable Zen errors:

```
import json as zenjson
pymport json

let bad = false
try {
    json.loads("{not json}")
} catch as e {
    bad = true
    print("caught json error")
}

if !bad {
    print("expected json.loads to fail")
    exit(1)
}

// Missing modules fail at the pymport statement and are catchable too
let missing = false
try {
    pymport this_module_does_not_exist
} catch as e {
    missing = true
    print("caught import error")
}

if !missing {
    print("expected import to fail")
    exit(1)
}

print("recovered: " + str(zenjson.parse("{\"ok\": true}")["ok"]))
```

14.8 42.8 Limits and Non-Goals (v1)

- **No keyword arguments** in `pymport` calls.
- **No passing Zen closures into Python** (rejected by marshal).
- **No per-handle finalizers**: foreign handles remain until bridge teardown.
- **Tuples → lists**, `big int` → `f64` in some cases, non-string dict keys rejected.
- **Functions containing `PyImport` not JIT-lowered** (stay interpreted).
- **No `zen.toml [py-deps]`** yet (use `requirements.txt`).
- **Network installs** only via `zen py-sync`/preamble; tests avoid silent installs.
- **Handles are opaque** across isolates in general.

14.9 42.9 Working Examples (tested)

All scripts live in `book-v2/tests/web-automation/`. Five smaller, hermetic pymport tests live in `book-v2/tests/pymport/`.

14.9.1 Example A: `bs4` + `requests` scrape + download

See `tests/web-automation/bs4_tiktok.z` — downloads a TikTok embed MP4 via `urllib.request.urlretrieve`, parsed with BeautifulSoup from `__FRONTITY_CONNECT_STATE__`.

14.9.2 Example B: `mechanicalsoup` stateful scrape + download

See `tests/web-automation/ms_tiktok.z`. Uses `StatefulBrowser`, accesses `browser.page`, reuses `browser.session.get()` for CDN, writes bytes via `builtins.open(...,"wb").write()`.

14.9.3 Example C: Selenium automation + download

See `tests/web-automation/selenium_tiktok.z`. Launches headless Chromium, polls for `__UNIVERSAL_DATA_FOR_REHYDRATION__`, extracts stats, fetches the embed route with plain HTTP for an unsigned MP4 URL, downloads via `urllib.request.urlretrieve`.

14.9.4 Example D: CDP over raw TCP/WebSocket (no Selenium)

See `tests/web-automation/cdp_tiktok.z`. Launches Chromium with `pymport subprocess`, discovers targets via HTTP `/json/list`, upgrades to WebSocket, sends masked CDP frames via `sys.write_bytes(socket.fileno(sock), hex)`, parses unmasked server frames, runs `Page.navigate` + `Runtime.evaluate` to extract TikTok metadata. Cleans up with `browser.terminate()` in a `finally` block.

Run all of them:

```
cd book-v2/tests
./run-all.sh --network
```

14.10 42.10 Troubleshooting

Issue	Fix
`Z0414` / “unknown native” / needs `--features python`	Rebuild with `cargo build --release --features python` and ensure libpython is available.
`ModuleNotFoundError` for third-party package	Install into venv, set `PYTHONPATH`, or run `zen py-sync` with `requirements.txt`.
Selenium can’t find chromedriver	Install `chromedriver` matching Chromium or put it on `PATH`. Use `--no-sandbox` in containers.
TikTok WAF blocks plain requests	Use `/embed/v2/<id>`, or Selenium/real browser. See Examples A/C.
Binary download corrupted	Don’t rely on string-based `fs.write_binary`; use `urllib.request.urlretrieve` or `builtins.open("wb").write()`.
WebSocket hang in raw sockets	Read until `Content-Length` for HTTP, handle EAGAIN on keep-alive, mask client frames, parse length (126/127).
`pymport` call with kwargs fails	Use positional args only in v1.
Bootstrap skipped unexpectedly	Check `ZEN_NO_PY_BOOTSTRAP` or that `requirements.txt` exists near entry.

14.11 42.11 References

- Docs: `docs/20-pymport.md` (design/spec)
- VM bridge: `crates/zen-vm/src/pybridge.rs`, `crates/zen-vm/src/pyembed.rs`
- CLI bootstrap: `zen-cli/src/pydeps.rs`
- Tests: `crates/zen-cli/tests/pymport_e2e.rs`, `crates/zen-vm/tests/pyembed.rs`, `crates/zen-jit/tests/jit_pymport.rs`
- Scripts: `book-v2/tests/web-automation/*.z`, `book-v2/tests/pymport/*.z`

14.12 42.12 Quick reference

```
pymport json
pymport os
pymport requests
pymport bs4
pymport selenium.webdriver as webdriver
pymport urllib.request
pymport builtins

// Positional calls only
```

```
json.loads('{"a": 1}')
os.getcwd()
os.path.join("a", "b")
datetime.datetime.now()
builtins.len([1, 2, 3])

// Error handling
try {
    pymport nonexistent_module
} catch as e {
    print("caught import error")
}

// zen py-sync setup
// ./target/release/zen py-sync
// ./target/release/zen run script.z
```

14.13 42.13 Exercises

1. Write a script that imports `json`, `os`, and `datetime` via `pymport` and prints the current date.
2. Use `pymport bs4` to parse an HTML file and extract all links.
3. Use `pymport selenium.webdriver` to open a headless browser and print the page title.
4. Write `py_len(x)` that calls `builtins.len(x)` on any value.
5. Demonstrate that `pymport` of a missing module is catchable with `try`/`catch`.

15 Chapter 44: Web Scraping & Automation

Web scraping and automation let you extract data from websites, download media, and control browsers programmatically. This chapter covers Zen's HTTP client, **pymport** (Python interoperability), browser automation with Selenium, and driving Chrome over **raw TCP/WebSocket sockets with the Chrome DevTools Protocol (CDP)**.

Every example here is runnable. The complete scripts, setup script, and test runner live in `book-v2/tests/web-automation/`; the pymport fundamentals are in Chapter 42.

> **Legal and ethical note:** Only scrape sites you are permitted to scrape. Check `robots.txt`, Terms of Service, and applicable laws. The examples target publicly accessible TikTok pages for educational purposes; sites may rate-limit, require authentication, or block automated access.

15.1 44.1 Setup

From the repository root:

```
cd book-v2/tests
./setup.sh
```

`setup.sh` does three things: 1. builds Zen with the Python bridge (`cargo build --release --features python`); 2. creates `.venv/` beside the tests; 3. installs `requirements.txt`.

Then, in the shell you use for the examples:

```
cd book-v2/tests/web-automation
export PYTHONPATH="$(../.venv/bin/python -c 'import sysconfig; print(sysconfig.get_path
export ZEN_NO_PY_BOOTSTRAP=1
ZEN=../../../target/release/zen
```

Requirements: - Zen built with `--features python` - Python 3.9+ with `venv` - `chromium` for Selenium and CDP - `chromedriver` matching Chromium for Selenium - Network access

Alternatively, let Zen manage a project environment: this directory has its own `requirements.txt`, so `zen py-sync` creates `.zen_venv/` and `zen run` uses it automatically. See Chapter 42.

> **pymport calls are positional-only.** No keyword arguments in v1. Pass `Options()` to `webdriver.Chrome(opts)` positionally, and pass `Popen` arguments positionally.

15.2 44.2 Run the whole suite

```
cd book-v2/tests
./run-all.sh              # hermetic pymport tests
./run-all.sh --network    # also runs the four examples below
```

The runner validates that each downloaded file is an MP4 (`ftyp` box) before reporting success.

15.3 44.3 1) Scrape + Download with `requests` + `bs4` (pymport)

`bs4` parses HTML. TikTok's embed page (`/embed/v2/<id>`) is accessible over plain HTTP and embeds video metadata in `__FRONTITY_CONNECT_STATE__`. The MP4 CDN URL is in `videoData.itemInfos.video.urls[0]`; `urllib.request.urlretrieve` writes it without re-encoding.

`bs4_tiktok.z`:

```
import requests
import fs
pymport bs4
pymport json
pymport urllib.request
pymport socket

socket.setdefaulttimeout(45)

let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let VIDEO = "6718335390845095173"
let url = "https://www.tiktok.com/embed/v2/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_bs4.mp4"

// TikTok's edge intermittently answers 503 to non-browser clients. Retry
// with backoff instead of failing the example on the first response.
let fetch = fn(u) {
    let attempt = 0
    let resp = null
    while attempt < 6 {
        resp = requests.get(u, {
            "headers": {
                "User-Agent": UA,
                "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=
                "Accept-Language": "en-US,en;q=0.9",
                "Referer": "https://www.tiktok.com/"
            },
            "timeout": 40
        })
        if resp["status_code"] == 200 {
            return resp
        }
        print("http " + str(resp["status_code"]) + " (attempt " + str(attempt + 1) + ")
        time.sleep(3 * (attempt + 1))
```

```
        attempt += 1
    }
    return resp
}

print("fetching " + url)
let resp = fetch(url)
print("status: " + str(resp["status_code"]) + " bytes: " + str(len(resp["text"])))
if resp["status_code"] != 200 {
    print("giving up: TikTok kept returning " + str(resp["status_code"]))
    exit(1)
}

let soup = bs4.BeautifulSoup(resp["text"], "html.parser")
let tag = soup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if tag == null {
    print("no state script")
    exit(1)
}

let data = json.loads(tag.string)
let page = data["source"]["data"]["/embed/v2/" + VIDEO]
let item = page["videoData"]["itemInfos"]
let author = page["videoData"]["authorInfos"]
let music = page["videoData"]["musicInfos"]
let video_url = item["video"]["urls"][0]

print("id: " + item["id"])
print("desc: " + item["text"])
print("author: @" + author["uniqueId"] + " (" + author["nickName"] + ")")
print("music: " + music["musicName"])
print("plays: " + str(item["playCount"]))
print("likes: " + str(item["diggCount"]))
print("comments: " + str(item["commentCount"]))
print("shares: " + str(item["shareCount"]))
print("mp4: " + video_url[0:80] + "...")

urllib.request.urlretrieve(video_url, out)
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")
```

Run:

```
$ZEN run bs4_tiktok.z
file out/tiktok_bs4.mp4
```

Expected: metadata for `@scout2015`, a `downloaded: out/tiktok_bs4.mp4 (3224978 bytes)` line, and `ISO Media, MP4 Base Media v1`.

15.4 44.4 2) Stateful Scraping + Download with `mechanicalsoup` (pymport)

`MechanicalSoup` wraps `requests` and BeautifulSoup in a stateful browser. `browser.open()` keeps cookies/history, `browser.page` is the current BS4 soup, and `browser.session` can be reused for the CDN fetch. Python `bytes` are written through Python's own file object because native `fs.write_binary` is string-oriented.

`ms_tiktok.z`:

```
import fs
import time
pymport mechanicalsoup
pymport json
pymport builtins
pymport socket

socket.setdefaulttimeout(45)

let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let VIDEO = "6718335390845095173"
let embed = "https://www.tiktok.com/embed/v2/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_ms.mp4"

let browser = mechanicalsoup.StatefulBrowser()
browser.session.headers.update({
    "User-Agent": UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Referer": "https://www.tiktok.com/"
})

let resp = browser.open(embed)
let attempt = 0
while resp.status_code != 200 && attempt < 6 {
    print("http " + str(resp.status_code) + " (attempt " + str(attempt + 1) + "), retry
    time.sleep(3 * (attempt + 1))
    resp = browser.open(embed)
    attempt += 1
}

print("opened: " + browser.get_url())
print("status: " + str(resp.status_code) + " bytes: " + str(len(resp.text)))
if resp.status_code != 200 {
    print("giving up: TikTok kept returning " + str(resp.status_code))
    exit(1)
}
```

```
let soup = browser.page
let tag = soup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if tag == null {
    print("no state script")
    exit(1)
}

let data = json.loads(tag.string)
let page = data["source"]["data"]["/embed/v2/" + VIDEO]
let item = page["videoData"]["itemInfos"]
let author = page["videoData"]["authorInfos"]
let video_url = item["video"]["urls"][0]

print("id: " + item["id"])
print("author: @" + author["uniqueId"] + " (" + author["nickName"] + ")")
print("plays: " + str(item["playCount"]) + " likes: " + str(item["diggCount"]))

let bin = browser.session.get(video_url)
print("cdn status: " + str(bin.status_code) + " bytes: " + str(len(bin.content)))
if bin.status_code != 200 {
    print("giving up: CDN returned " + str(bin.status_code))
    exit(1)
}

let f = builtins.open(out, "wb")
f.write(bin.content)
f.close()
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")
```

Run:

```
$ZEN run ms_tiktok.z
```

15.5 44.5 3) Browser Automation with Selenium (pymport)

Headless Chromium solves JavaScript challenges and renders pages like a user. Selenium navigates to the real TikTok video page, waits for `__UNIVERSAL_DATA_FOR_REHYDRATION__`, and extracts the full `itemStruct` (author, stats, description). The video page's `playAddr` is signed and rejected by the CDN outside the player, so the script fetches the embed route with plain HTTP for an unsigned MP4 URL.

`selenium_tiktok.z`:

```
import fs
import time
import requests
pymport selenium.webdriver as webdriver
pymport selenium.webdriver.chrome.options as options
```

```
pymport bs4
pymport json
pymport urllib.request
pymport socket

socket.setdefaulttimeout(45)

let VIDEO = "6718335390845095173"
let UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gec
let url = "https://www.tiktok.com/@scout2015/video/" + VIDEO

let _ = fs.mkdirs("out")
let out = "out/tiktok_selenium.mp4"

// Quit the browser and stop the chromedriver service. The embedded
// interpreter exits immediately, so this must happen before exit().
let close_browser = fn(d) {
    d.quit()
    try {
        d.service.stop()
    } catch {
        # already stopped
    }
}

let opts = options.Options()
opts.add_argument("--headless=new")
opts.add_argument("--no-sandbox")
opts.add_argument("--disable-dev-shm-usage")
opts.add_argument("--disable-gpu")
opts.add_argument("--window-size=1280,900")
opts.add_argument("--mute-audio")

print("launching chromium...")
let driver = webdriver.Chrome(opts)
driver.set_page_load_timeout(60)

print("navigating to " + url)
try {
    driver.get(url)
} catch as e {
    print("navigation warning: " + str(e))
}

// The page renders its data after the WAF/JS boot; poll for the payload.
let src = ""
let tries = 0
```

```
while tries < 15 {
    src = driver.page_source
    if src.contains("__UNIVERSAL_DATA_FOR_REHYDRATION__") {
        break
    }
    time.sleep(2)
    tries += 1
}
print("title: " + driver.title)
print("page bytes: " + str(len(src)) + " (polls: " + str(tries) + ")")

let soup = bs4.BeautifulSoup(src, "html.parser")
let tag = soup.find("script", {"id": "__UNIVERSAL_DATA_FOR_REHYDRATION__"})
if tag == null {
    print("no rehydration payload - TikTok served a challenge")
    close_browser(driver)
    exit(1)
}

let data = json.loads(tag.string)
let item = data["__DEFAULT_SCOPE__"]["webapp.video-detail"]["itemInfo"]["itemStruct"]
let stats = item["stats"]
let author = item["author"]

print("id: " + item["id"])
print("desc: " + item["desc"])
print("author: @" + author["uniqueId"] + " (" + author["nickname"] + ")")
print("plays: " + str(stats["playCount"]) + " likes: " + str(stats["diggCount"]))
print("comments: " + str(stats["commentCount"]) + " shares: " + str(stats["shareCount"]

print("fetching embed route for an unsigned mp4 url...")
let embed = "https://www.tiktok.com/embed/v2/" + VIDEO
let embed_headers = {
    "User-Agent": UA,
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
    "Accept-Language": "en-US,en;q=0.9",
    "Referer": "https://www.tiktok.com/"
}
let eresp = null
let eretry = 0
while eretry < 6 {
    eresp = requests.get(embed, {"headers": embed_headers, "timeout": 40})
    if eresp["status_code"] == 200 {
        break
    }
    print("embed http " + str(eresp["status_code"]) + " (attempt " + str(eretry + 1) +
    time.sleep(3 * (eretry + 1))
```

```
    eretry += 1
}
print("embed status: " + str(eresp["status_code"]))
if eresp["status_code"] != 200 {
    close_browser(driver)
    exit(1)
}
let esoup = bs4.BeautifulSoup(eresp["text"], "html.parser")
let etag = esoup.find("script", {"id": "__FRONTITY_CONNECT_STATE__"})
if etag == null {
    print("no embed state script")
    close_browser(driver)
    exit(1)
}
let edata = json.loads(etag.string)
let epage = edata["source"]["data"]["/embed/v2/" + VIDEO]
let mp4 = epage["videoData"]["itemInfos"]["video"]["urls"][0]
print("mp4: " + mp4[0:80] + "...")

urllib.request.urlretrieve(mp4, out)
print("downloaded: " + out + " (" + str(fs.size(out)) + " bytes)")

close_browser(driver)
time.sleep(1)
print("browser closed")
```

Run (Chromium startup can take 30-40s):

```
timeout 300 $ZEN run selenium_tiktok.z
```

Chromedriver must match the Chromium major version. On Debian/Ubuntu: `apt install chromium-driver`.

15.6 44.6 4) Low-Level Control: CDP over raw TCP + WebSocket sockets

This example uses no automation library. Zen launches Chromium with `--remote-debugging-port`, discovers the page target over HTTP (`/json/list`), upgrades to a WebSocket, sends masked CDP frames through `sys.write_bytes` on the socket fd, and parses unmasked server frames byte by byte.

Key protocol details: - **HTTP**: parse headers and stop at `Content-Length`. Chrome may keep the connection open despite `Connection: close`; waiting for EOF produces a timeout (`EAGAIN`). - **WebSocket**: client frames must be masked. Extended lengths are 126 + 16-bit or 127 + 64-bit. Server frames are not masked. - **Binary writes**: `socket.send` takes a string and would UTF-8 re-encode masked bytes. `sys.write_bytes(fd, hex)` writes raw bytes, so the frame is built as a byte list, hex-encoded, and written to `socket.fileno(sock)`.

cdp_tiktok.z helpers:

```
// CDP over raw sockets: drive headless Chromium from Zen with no libraries.
import socket
import sys
import json
import time
import fs
pymport subprocess
import base64
import bytes

let PORT = 9223
let PROFILE = "out/cdp-profile"
let VIDEO = "6718335390845095173"
let VIDEO_PAGE = "https://www.tiktok.com/@scout2015/video/" + VIDEO

let HEX = "0123456789abcdef"

let to_hex = fn(bs) {
    let out = ""
    for b in bs {
        out += HEX[b >> 4] + HEX[b & 15]
    }
    return out
}

let ws_send = fn(sock, text) {
    let payload = []
    for ch in list(text) {
        for b in bytes.utf8_bytes(ch.ord()) {
            payload.push(b)
        }
    }
    let mask = [0x37, 0xA1, 0x0C, 0xF2]
    let masked = []
    let i = 0
    while i < len(payload) {
        masked.push(payload[i] ^ mask[i % 4])
        i += 1
    }
    let frame = [0x81]
    let n = len(masked)
    if n < 126 {
        frame.push(0x80 | n)
    } elif n < 65536 {
        frame.push(0x80 | 126)
        frame.push((n >> 8) & 255)
```

```
        frame.push(n & 255)
    } else {
        frame.push(0x80 | 127)
        let k = 7
        while k >= 0 {
            frame.push((n >> (8 * k)) & 255)
            k -= 1
        }
    }
    for b in mask {
        frame.push(b)
    }
    for b in masked {
        frame.push(b)
    }
    sys.write_bytes(socket.fileno(sock), to_hex(frame))
}

let recv_exact = fn(sock, n) {
    let out = []
    while len(out) < n {
        let chunk = list(socket.recv(sock, n - len(out)))
        if len(chunk) == 0 {
            throw "websocket closed"
        }
        for b in chunk {
            out.push(b)
        }
    }
    return out
}

let ws_read = fn(sock) {
    let h = recv_exact(sock, 2)
    let opcode = h[0] & 0x0F
    let masked = (h[1] & 0x80) != 0
    let n = h[1] & 0x7F
    if n == 126 {
        let e = recv_exact(sock, 2)
        n = e[0] * 256 + e[1]
    } elif n == 127 {
        let e = recv_exact(sock, 8)
        n = 0
        for b in e {
            n = n * 256 + b
        }
    }
```

```
    let mask = []
    if masked {
        mask = recv_exact(sock, 4)
    }
    let data = recv_exact(sock, n)
    if masked {
        let i = 0
        while i < len(data) {
            data[i] = data[i] ^ mask[i % 4]
            i += 1
        }
    }
    return {"opcode": opcode, "data": data}
}

let cdp_call = fn(sock, id, method, params) {
    ws_send(sock, json.stringify({"id": id, "method": method, "params": params}))
    let pending = []
    while true {
        let f = ws_read(sock)
        if f["opcode"] == 8 {
            throw "websocket closed by peer"
        }
        if f["opcode"] == 9 {
            continue
        }
        if f["opcode"] == 1 or f["opcode"] == 0 {
            for b in f["data"] {
                pending.push(b)
            }
            let text = bytes.str_from_bytes(pending)
            try {
                let msg = json.parse(text)
                if msg.get("id", 0) == id {
                    return msg
                }
                pending = []
            } catch {
                # fragmented frame: keep collecting
            }
        }
    }
}
```

The full script then launches Chromium with `pymport subprocess`, performs the HTTP discovery and WebSocket upgrade, and polls `Runtime.evaluate`:

```
let _ = fs.mkdirs("out")
let _ = fs.rmtree(PROFILE)
let cmd = "chromium --headless=new --no-sandbox --disable-gpu --disable-dev-shm-usage "
cmd += "--remote-debugging-port=" + str(PORT) + " "
cmd += "--user-data-dir=" + PROFILE + " about:blank"
let argv = cmd.split(" ")
let browser = subprocess.Popen(argv, -1, null, null, null, -3)
print("chromium pid " + str(browser.pid) + " on port " + str(PORT))

try {
    # ... HTTP discovery, WS upgrade ...
    let nav = cdp_call(ws, 1, "Page.navigate", {"url": VIDEO_PAGE})
    let reply = cdp_call(ws, 100, "Runtime.evaluate", {"expression": expr, "returnByVal
    # expr returns id, author, plays, likes, comments, desc
} finally {
    browser.terminate()
}
```

See the complete, tested script: tests/web-automation/cdp_tiktok.z.

Run:

```
timeout 180 $ZEN run cdp_tiktok.z
```

15.7 44.7 Tips and Gotchas

- **WAF challenges**: plain `requests` to TikTok video/profile pages may return a WAF challenge. Use `/embed/v2/<id>` or a real browser (Selenium). The embed route is reliable for the MP4 URL.
- **Binary downloads**: `fs.write_binary` expects a string and has UTF-8/string constraints. Prefer `urllib.request.urlretrieve` or `builtins.open(..., "wb").write(py_bytes)`.
- **pymport limits**: positional-only calls (no kwargs). `webdriver.Chrome(opts)` and `Popen(argv, -1, null, null, null, -3)` are positional.
- **Environment bootstrap**: use `book-v2/tests/setup.sh`, or `zen py-sync` with a local `requirements.txt`.
- **Chrome sandbox**: in containers, add `--no-sandbox` and `--disable-dev-shm-usage`.
- **Sockets**: `socket.recv` returns `bytes`; use `list(bytes)` to index. `socket.recv_text` returns a string and is lossy for binary. For raw binary protocols, write through `sys.write_bytes(fd, hex)`.
- **JSON**: the pure-Zen `std/json.z` stringifies numbers with `str()`; nested dict access in CDP responses needs explicit keys.
- **Cleanup**: Selenium can leave `chromedriver` alive when the embedded interpreter exits; the example calls `driver.service.stop()`. For a hard-killed CDP run, remove leftovers with `pkill -x chromium`.
- **Hangs**: `MechanicalSoup`, `urllib.request.urlretrieve`, and `requests` do not impose a default timeout. Set `socket.setdefaulttimeout(45)` (via `pympor socket`)

or pass an explicit timeout where supported.

15.8 44.8 Verified Artifacts

Tested against real TikTok on 2026-10-23:

- `out/tiktok_bs4.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- `out/tiktok_ms.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- `out/tiktok_selenium.mp4` — ISO Media, MP4 Base Media v1 (3,224,978 bytes)
- CDP over raw sockets extracted metadata from `@scout2015/video/6718335390845095173`: 159600 plays, 35200 likes, 5486 comments.

15.9 44.9 Appendix: Tested Scripts

Everything in this chapter is in the repository:

- `tests/web-automation/bs4_tiktok.z` — requests + bs4 + urllib download
- `tests/web-automation/ms_tiktok.z` — MechanicalSoup + `browser.session` + `builtins` binary write
- `tests/web-automation/selenium_tiktok.z` — Selenium headless + embed route
- `tests/web-automation/cdp_tiktok.z` — raw TCP + WebSocket CDP
- `tests/web-automation/requirements.txt`
- `tests/setup.sh` — build + venv + install
- `tests/run-all.sh` — run and validate

Quick start:

```
git clone https://github.com/3CN-O4D/zen.git
cd zen/book-v2/tests
./setup.sh
./run-all.sh --network
```

See Chapter 42: pymport for setup, `zen py-sync`, and troubleshooting.

16 Chapter 45: REST API Client

This chapter shows how to build a REST API client in Zen using `requests`, `json`, and `argparser`. It covers authentication, pagination, rate limiting, error handling, and building a reusable client class.

16.1 45.1 Basic GET request

```
import requests
import json

let resp = requests.get("https://api.example.com/users/1")
if resp.ok {
    let user = resp.json()
    print(user["name"])
} else {
    print("HTTP " + str(resp.status_code))
}
```

16.2 45.2 POST with JSON body

```
import requests
import json

let body = json.dumps({"name": "Zen", "version": 3})
let resp = requests.post("https://example.com/api/users", body, {
    "headers": {"Content-Type": "application/json"}
})
print(resp.status_code)
print(resp.text)
```

16.3 45.3 Reusable API client

```
import requests
import json

class ApiClient {
    let base_url
    let headers
    let session

    func init(base_url, headers = {}) {
        this.base_url = base_url
        this.headers = headers
        this.session = requests.Session()
        this.session.headers = headers
```

```
    }

    func get(path, params = {}) {
        let url = this.base_url + path
        let resp = requests.sess_get(this.session, url, {"params": params})
        return resp
    }

    func post(path, data) {
        let url = this.base_url + path
        let body = json.dumps(data)
        let resp = requests.sess_post(this.session, url, {
            "json": body,
            "headers": {"Content-Type": "application/json"}
        })
        return resp
    }

    func put(path, data) {
        let url = this.base_url + path
        let body = json.dumps(data)
        let resp = requests.sess_put(this.session, url, {
            "json": body,
            "headers": {"Content-Type": "application/json"}
        })
        return resp
    }

    func delete(path) {
        let url = this.base_url + path
        let resp = requests.sess_delete(this.session, url)
        return resp
    }

    func close() {
        requests.session_close(this.session)
    }
}
```

16.3.1 Usage

```
let client = new ApiClient("https://api.example.com", {
    "Authorization": "Bearer token123",
    "User-Agent": "ZenClient/1.0"
})
```

```
let resp = client.get("/users/1")
if resp.ok {
    print(resp.json()["name"])
}

let created = client.post("/users", {"name": "New User"})
print(created.status_code)

client.close()
```

16.4 45.4 Authentication

16.4.1 Bearer token

```
let client = new ApiClient("https://api.example.com", {
    "Authorization": "Bearer " + token
})
```

16.4.2 Basic auth

```
import requests

let resp = requests.get("https://example.com/api", {
    "auth": ["user", "pass"]
})
```

16.4.3 API key header

```
let client = new ApiClient("https://api.example.com", {
    "X-API-Key": "your-api-key"
})
```

16.5 45.5 Pagination

Many APIs paginate results. Here's a pattern for following `next` links:

```
import requests

func get_all_pages(url) {
    let all_items = []
    let next_url = url
    while next_url != nil {
        let resp = requests.get(next_url)
        if !resp.ok {
            break
        }
```

```
        let data = resp.json()
        for item in data["items"] {
            all_items.push(item)
        }
        next_url = data["next"]  // null when no more pages
    }
    return all_items
}

let items = get_all_pages("https://api.example.com/items?page=1")
print(len(items))
```

16.5.1 Offset-based pagination

```
func get_all_offset(base_url, page_size = 50) {
    let all_items = []
    let offset = 0
    while true {
        let resp = requests.get(base_url, {
            "params": {"offset": offset, "limit": page_size}
        })
        if !resp.ok {
            break
        }
        let data = resp.json()
        let items = data["items"]
        for item in items {
            all_items.push(item)
        }
        if len(items) < page_size {
            break
        }
        offset = offset + page_size
    }
    return all_items
}
```

16.6 45.6 Rate limiting

Respect `RateLimit` headers and retry on 429:

```
import requests
import time

func get_with_retry(url, max_retries = 3) {
    let attempt = 0
    while attempt < max_retries {
```

```
        let resp = requests.get(url)
        if resp.ok {
            return resp
        }
        if resp.status_code == 429 {
            let retry_after = resp.headers["Retry-After"]
            let wait = retry_after != nil ? int(retry_after) : 1
            print("Rate limited, waiting " + str(wait) + "s")
            time.sleep(wait)
            attempt = attempt + 1
        } else {
            break
        }
    }
    return nil
}

let resp = get_with_retry("https://api.example.com/data")
if resp != nil {
    print(resp.text)
}
```

16.7 45.7 Error handling

```
import requests

func api_get(client, path) {
    let resp = client.get(path)
    if resp.status_code == 404 {
        return nil  // not found
    }
    if !resp.ok {
        throw "API error: " + str(resp.status_code) + " " + resp.reason
    }
    return resp.json()
}

try {
    let user = api_get(client, "/users/999")
    if user == nil {
        print("User not found")
    } else {
        print(user["name"])
    }
} catch as e {
    print("API call failed: " + e.msg)
```

```
}
```

16.8 45.8 Complete CLI example

```
import requests
import json
import argparser
import fs

let parser = argparser.ArgumentParser("api-client")
parser.add_argument("url", help="API URL")
parser.add_argument("--method", "-m", default="GET", help="HTTP method")
parser.add_argument("--data", "-d", help="JSON body")
parser.add_argument("--header", "-H", action="append", help="Header (key:value)")
let args = parser.parse_args()

let headers = {}
if args.header != nil {
    for h in args.header {
        let parts = h.split(":", 1)
        if len(parts) == 2 {
            headers[parts[0]] = parts[1]
        }
    }
}

let body = args.data != nil ? args.data : nil

let resp = requests.get(args.url, {"headers": headers})
// ... extend for other methods

print(resp.status_code)
print(resp.text)
```

16.9 45.9 Common pitfalls

- **Always check `resp.ok`.** A non-200 response still returns a response object.
- **JSON bodies must be strings.** Use `json.dumps()` to serialize.
- **Sessions persist cookies and headers.** Close them with `requests.session_close()`.
- **Rate limits.** Respect `Retry-After` and back off on 429.
- **Timeouts.** Always pass a `timeout` option to avoid hanging.

16.10 45.10 Quick reference

```
import requests
import json
```

```
// GET
let resp = requests.get("https://api.example.com/items", {
    "params": {"page": 1},
    "headers": {"Authorization": "Bearer token"}
})

// POST JSON
let resp = requests.post("https://example.com/api", {
    "json": json.dumps({"name": "Zen"}),
    "headers": {"Content-Type": "application/json"}
})

// Session
let sess = requests.Session()
sess.headers = {"Authorization": "Bearer token"}
let resp = requests.sess_get(sess, "https://example.com/api")
requests.session_close(sess)

// Status
resp.status_code
resp.ok
resp.text
resp.json()
resp.headers
```

16.11 45.11 Exercises

1. Write a REST client for a public API (e.g., JSONPlaceholder) that lists posts.
2. Add pagination support to fetch all pages.
3. Add retry logic with exponential backoff.
4. Write a CLI that takes a URL and method and prints the response.
5. Build a small config client that reads/writes JSON config via REST.

17 Chapter 46: File Processing Tools

This chapter shows how to process files in Zen: reading/writing text and binary, CSV, JSON, line-by-line processing, temporary files, and directory walking.

17.1 46.1 Reading text files

```
import fs

// Read entire file
let content = fs.read("data.txt")
print(content)

// Read line by line
let lines = fs.read("data.txt").split("\n")
for line in lines {
    if len(line) > 0 {
        print(line)
    }
}
```

17.2 46.2 Writing text files

```
import fs

// Write entire file (overwrites)
fs.write("output.txt", "Line 1\nLine 2\nLine 3\n")

// Append (read-modify-write)
let existing = ""
if fs.exists("log.txt") {
    existing = fs.read("log.txt")
}
fs.write("log.txt", existing + "new entry\n")
```

17.3 46.3 Binary files

```
import fs

// Read binary
let data = fs.read_binary("image.png")
print(len(data))   // byte count

// Write binary
fs.write_binary("copy.png", data)
```

```
// Copy a file
fs.copy("source.txt", "destination.txt")
```

17.4 46.4 CSV processing (manual)

```
import fs

func read_csv(path) {
    let lines = fs.read(path).split("\n")
    if len(lines) == 0 {
        return []
    }
    let header = lines[0].split(",")
    let rows = []
    for i in 1..len(lines) {
        if lines[i] == "" {
            continue
        }
        let values = lines[i].split(",")
        let row = {}
        for j in 0..len(header) {
            row[header[j]] = values[j]
        }
        rows.push(row)
    }
    return rows
}

let data = read_csv("data.csv")
for row in data {
    print(row["name"])
}
```

17.5 46.5 JSON files

```
import fs
import json

// Read JSON
let text = fs.read("config.json")
let config = json.loads(text)
print(config["theme"])

// Write JSON
let data = {"name": "Zen", "version": 3}
fs.write("config.json", json.dumps(data))
```

17.6 46.6 Line processing pipeline

```
import fs

func process_lines(path, func) {
    let lines = fs.read(path).split("\n")
    let results = []
    for line in lines {
        if len(line) > 0 {
            results.push(func(line))
        }
    }
    return results
}

let upper_lines = process_lines("data.txt", fn(line) {
    return line.upper()
})
for line in upper_lines {
    print(line)
}
```

17.7 46.7 Temporary files

```
import tempfile
import fs

let tmp = tempfile.mktemp()
fs.write(tmp, "temporary data")
let content = fs.read(tmp)
fs.remove(tmp)

// Or use a directory
let dir = tempfile.mkdtemp()
// ... use dir ...
fs.rmtree(dir)
```

17.8 46.8 Directory walking

```
import fs

func walk(dir, func) {
    for name in fs.list(dir) {
        let path = fs.join(dir, name)
```

```
        if fs.is_dir(path) {
            walk(path, func)
        } else {
            func(path)
        }
    }
}

walk(".", fn(path) {
    print(path)
})
```

17.9 46.9 File info

```
import fs

print(fs.exists("file.txt"))      # true/false
print(fs.is_file("file.txt"))     # true/false
print(fs.is_dir("dir"))           # true/false
print(fs.size("file.txt"))        // byte count
print(fs.join("a", "b", "c"))     // a/b/c
print(fs.basename("/a/b/c.txt"))  // c.txt
print(fs.dirname("/a/b/c.txt"))   // /a/b
```

17.10 46.10 Quick reference

```
import fs
import json
import tempfile

fs.read("file.txt")           # string
fs.read_binary("file.bin")    # bytes
fs.write("file.txt", "data")
fs.write_binary("file.bin", bytes)
fs.exists("file.txt")
fs.is_file("file.txt")
fs.is_dir("dir")
fs.size("file.txt")
fs.mkdirs("a/b/c")
fs.remove("file.txt")
fs.rmtree("dir")
fs.copy("src", "dst")
fs.move("old", "new")
fs.join("a", "b")
fs.basename("/a/b/c.txt")
fs.dirname("/a/b/c.txt")
```

```
fs.list(".")                  # directory entries

let tmp = tempfile.mktemp()
fs.write(tmp, "data")
fs.remove(tmp)

let dir = tempfile.mkdtemp()
fs.rmtree(dir)
```

17.11 46.11 Exercises

1. Write `file_size_human(path)` that returns a human-readable file size.
2. Write a script that copies all `.txt` files from one directory to another.
3. Write `grep_file(pattern, path)` that prints lines containing `pattern`.
4. Write a file tailer that reads the last `n` lines of a file.
5. Write `csv_to_json(path)` that converts a CSV file to JSON.
6. Write `count_lines(path)` that counts non-empty lines in a file.
7. Write `merge_files(paths, output)` that concatenates multiple files into one.

18 Chapter 47: Network Security Tools

Note: This chapter is for educational and ethical purposes only. Always get proper authorization before testing network security.

18.1 47.1 Port Scanner

```
import socket

func scan_port(host, port, timeout) {
    try {
        let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
        socket.settimeout(sock, timeout)
        socket.connect(sock, host, port)
        socket.close(sock)
        return true
    } catch {
        return false
    }
}

func scan_ports(host, start_port, end_port) {
    print("Scanning " + host + " from port " + str(start_port) + " to " + str(end_port)
    let open_ports = []
    let port = start_port
    while port <= end_port {
        if scan_port(host, port, 500) {
            print("Port " + str(port) + " is OPEN")
            open_ports.push(port)
        }
        port = port + 1
    }
    return open_ports
}

// Usage (only scan hosts you own or have permission to scan)
// let open = scan_ports("127.0.0.1", 1, 1024)
```

18.2 44. Subdomain Finder

```
import dns
import requests

func find_subdomains(domain, wordlist) {
    let found = []
    for sub in wordlist {
```

```
        let hostname = sub + "." + domain
        try {
            let ips = dns.resolve(hostname)
            print("Found: " + hostname + " -> " + ips[0])
            found.push({"subdomain": hostname, "ips": ips})
        } catch {
            # Not found
        }
    }
    return found
}

let wordlist = ["www", "mail", "ftp", "api", "admin", "test", "dev"]
// find_subdomains("example.com", wordlist)
```

18.3 44. HTTP Header Checker

```
import requests

func check_headers(url) {
    try {
        let resp = requests.get(url)
        print("Security Headers for " + url)
        print("=" * 40)

        let security_headers = [
            "Strict-Transport-Security",
            "Content-Security-Policy",
            "X-Frame-Options",
            "X-Content-Type-Options",
            "Referrer-Policy"
        ]

        for h in security_headers {
            let val = resp.headers[h]
            if val != nil {
                print(h + ": " + val)
            } else {
                print(h + ": MISSING")
            }
        }
    } catch as e {
        print("Error: " + e.msg)
    }
}
```

```
// check_headers("https://example.com")
```

18.4 44.5 Port Scan Report

```
import socket
import fs

func scan_and_report(host, start, end, output_file) {
    let open_ports = []
    for port in start..end {
        try {
            let sock = socket.create(socket.AF_INET, socket.SOCK_STREAM)
            socket.settimeout(sock, 300)
            socket.connect(sock, host, port)
            socket.close(sock)
            open_ports.push(port)
            print("OPEN: " + str(port))
        } catch {
            # closed or filtered
        }
    }

    let report = "# Scan Report for " + host + "\n"
    report = report + "Open ports: " + str(len(open_ports)) + "\n"
    for p in open_ports {
        report = report + "  - " + str(p) + "\n"
    }

    fs.write(output_file, report)
    print("Report written to " + output_file)
}

// scan_and_report("127.0.0.1", 1, 1000, "scan_report.txt")
```

18.5 44.6 HTTP Header Analysis

```
import requests

func analyze_security(url) {
    try {
        let resp = requests.get(url, {"timeout": 10})

        print("URL: " + url)
        print("Status: " + str(resp.status_code))
        print("")
```

```
        let headers = [
            "Strict-Transport-Security",
            "Content-Security-Policy",
            "X-Frame-Options",
            "X-Content-Type-Options",
            "X-XSS-Protection",
            "Referrer-Policy",
            "Permissions-Policy"
        ]

        for h in headers {
            let val = resp.headers[h]
            if val == nil {
                print("[MISSING] " + h)
            } else {
                print("[OK] " + h + ": " + val)
            }
        }

        // Check for server header disclosure
        let server = resp.headers["Server"]
        if server != nil {
            print("[INFO] Server header: " + server)
        }

    } catch as e {
        print("Error: " + e.msg)
    }
}

// analyze_security("https://example.com")
```

18.6 44.7 DNS Reconnaissance

```
import dns

func dns_recon(domain) {
    print("DNS Recon for " + domain)
    print("")

    // A record
    try {
        let addrs = dns.resolve(domain)
        print("A records:")
        for a in addrs {
            print("  " + a)
```

```
        }
    } catch {
        print("No A records found")
    }

    // MX record (if supported)
    // ...

    print("")
    print("FQDN: " + dns.getfqdn(domain))
}

// dns_recon("example.com")
```

18.7 44.8 Common pitfalls

- **Only scan hosts you own or have explicit written permission to test.**
- **Rate-limit your requests** to avoid overwhelming the target.
- **TLS certificate errors** may occur with self-signed certs.
- **Some WAFs block automated scanning.**

18.8 44.9 Quick reference

```
import socket
import dns
import requests

// Port scan
socket.create(socket.AF_INET, socket.SOCK_STREAM)
socket.connect(sock, host, port)
socket.close(sock)

// DNS resolve
dns.resolve(domain)

// HTTP headers
requests.get(url)
resp.headers[header_name]
```

18.9 44.10 Exercises

1. Write a port scanner that scans common ports (21, 22, 25, 53, 80, 443, 3306, 3389).
2. Write a subdomain enumerator with a custom wordlist.
3. Write a security header checker for a list of URLs.
4. Write a DNS reconnaissance script that gathers A, MX, and NS records.

5. Write a scan report generator that saves results to a file.

19 Chapter 48: Capstone Projects

This chapter presents complete, end-to-end projects that combine multiple Zen features: pymport, web scraping, file processing, security tools, and CLI building.

19.1 48.1 Capstone 1: Web Metadata Archiver

A tool that fetches a webpage, extracts metadata (title, description, images, links), and saves it as a JSON file.

```
import requests
import bs4
import json
import fs
import argparser
import sys

func archive_metadata(url, output_file) {
    let resp = requests.get(url, {
        "headers": {"User-Agent": "Zen-Archiver/1.0"},
        "timeout": 30
    })

    if !resp.ok {
        throw "Failed to fetch " + url + ": HTTP " + str(resp.status_code)
    }

    let soup = bs4.BeautifulSoup(resp["text"], "html.parser")

    let metadata = {
        "url": url,
        "title": "",
        "description": "",
        "images": [],
        "links": []
    }

    // Title
    let title_tag = soup.find("title")
    if title_tag != null {
        metadata["title"] = title_tag.string
    }

    // Meta description
    let desc_tag = soup.find("meta", {"name": "description"})
    if desc_tag != null {
        metadata["description"] = desc_tag.get("content", "")
    }
```

```
    // Images
    let img_tags = soup.find_all("img")
    for img in img_tags {
        let src = img.get("src", "")
        if len(src) > 0 {
            metadata["images"].push(src)
        }
    }

    // Links
    let link_tags = soup.find_all("a")
    for link in link_tags {
        let href = link.get("href", "")
        if len(href) > 0 {
            metadata["links"].push(href)
        }
    }

    fs.write(output_file, json.dumps(metadata, {"indent": 2}))
    print("Metadata saved to " + output_file)
    print("Title: " + metadata["title"])
    print("Images: " + str(len(metadata["images"])))
    print("Links: " + str(len(metadata["links"])))
}

let parser = argparser.ArgumentParser("archive-metadata")
parser.add_argument("url", help="URL to archive")
parser.add_argument("--output", "-o", default="metadata.json", help="Output file")
let args = parser.parse_args()

try {
    archive_metadata(args.url, args.output)
} catch as e {
    fs.write("/dev/stderr", "Error: " + e.msg + "\n")
    sys.exit(1)
}
```

19.2 48.2 Capstone 2: Password Strength Checker

A tool that checks password strength using entropy calculation and common password lists.

```
import fs
import math
import random
import argparser
```

```
func password_strength(password) {
    let score = 0
    let reasons = []

    let length = len(password)
    if length >= 8 {
        score = score + 1
    } else {
        reasons.push("Password too short (min 8)")
    }

    let has_lower = false
    let has_upper = false
    let has_digit = false
    let has_special = false

    for ch in password {
        let code = ord(ch)
        if code >= 97 and code <= 122 {
            has_lower = true
        } elif code >= 65 and code <= 90 {
            has_upper = true
        } elif code >= 48 and code <= 57 {
            has_digit = true
        } else {
            has_special = true
        }
    }

    if has_lower { score = score + 1 } else { reasons.push("Add lowercase") }
    if has_upper { score = score + 1 } else { reasons.push("Add uppercase") }
    if has_digit { score = score + 1 } else { reasons.push("Add digit") }
    if has_special { score = score + 1 } else { reasons.push("Add special char") }

    let strength = ["Very Weak", "Weak", "Fair", "Good", "Strong"]
    let level = strength[min(score, 4)]

    return {"score": score, "level": level, "reasons": reasons}
}

func generate_password(length) {
    let chars = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#$%^&*
    let pw = []
    for i in 1..length {
        pw.push(random.choice(chars))
    }
    return "".join(pw)
```

```
}

let parser = argparser.ArgumentParser("password-checker")
parser.add_argument("password", nargs="?", help="Password to check")
parser.add_argument("--generate", "-g", type="int", help="Generate random password of l
let args = parser.parse_args()

if args.generate != nil {
    let pw = generate_password(args.generate)
    let strength = password_strength(pw)
    print("Generated: " + pw)
    print("Strength: " + strength["level"] + " (" + str(strength["score"]) + "/5)")
} elif args.password != nil {
    let strength = password_strength(args.password)
    print("Password: " + args.password)
    print("Strength: " + strength["level"] + " (" + str(strength["score"]) + "/5)")
    if len(strength["reasons"]) > 0 {
        print("Suggestions:")
        for r in strength["reasons"] {
            print("  - " + r)
        }
    }
} else {
    parser.print_help()
}
```

19.3 48.3 Capstone 3: Log Analyzer

A tool that parses web server logs and produces summary statistics.

```
import fs
import re
import argparser

func analyze_log(path) {
    let content = fs.read(path)
    let lines = content.split("\n")

    let total = 0
    let status_codes = {}
    let ips = {}
    let paths = {}

    let log_re = re.compile("^(\\S+) \\S+ \\S+ \\[.*\\] \"(\\S+) (\\S+) \\S+\" (\\d{3})

    for line in lines {
        if len(line) == 0 {
```

```
            continue
        }
        total = total + 1

        let m = log_re.search(line)
        if m != nil {
            let ip = m.group(1)
            let method = m.group(2)
            let path = m.group(3)
            let status = m.group(4)

            ips[ip] = (ips.get(ip, 0) + 1)
            paths[path] = (paths.get(path, 0) + 1)
            status_codes[status] = (status_codes.get(status, 0) + 1)
        }
    }

    return {
        "total": total,
        "status_codes": status_codes,
        "top_ips": sort_by_value(ips, 10),
        "top_paths": sort_by_value(paths, 10)
    }
}

func sort_by_value(dict, n) {
    let items = []
    for k in dict.keys() {
        items.push([k, dict[k]])
    }
    // Simple bubble sort by value descending
    for i in 0..len(items) {
        for j in 0..(len(items) - i - 1) {
            if items[j][1] < items[j + 1][1] {
                let tmp = items[j]
                items[j] = items[j + 1]
                items[j + 1] = tmp
            }
        }
    }
    let result = []
    for i in 0..min(n, len(items)) {
        result.push(items[i])
    }
    return result
}
```

```
let parser = argparser.ArgumentParser("log-analyzer")
parser.add_argument("file", help="Log file to analyze")
let args = parser.parse_args()

try {
    let result = analyze_log(args.file)
    print("Log Analysis Report")
    print("=" * 40)
    print("Total requests: " + str(result["total"]))
    print("")
    print("Status codes:")
    for code in result["status_codes"].keys() {
        print("  " + code + ": " + str(result["status_codes"][code]))
    }
    print("")
    print("Top IPs:")
    for item in result["top_ips"] {
        print("  " + item[0] + ": " + str(item[1]))
    }
    print("")
    print("Top paths:")
    for item in result["top_paths"] {
        print("  " + item[0] + ": " + str(item[1]))
    }
} catch as e {
    fs.write("/dev/stderr", "Error: " + e.msg + "\n")
    sys.exit(1)
}
```

19.4 48.4 Capstone 4: Site Uptime Monitor

A tool that checks multiple URLs and reports which are down.

```
import requests
import fs
import argparser
import time

func check_url(url, timeout) {
    try {
        let resp = requests.get(url, {"timeout": timeout})
        return {
            "url": url,
            "status": resp.status_code,
            "ok": resp.ok,
            "error": nil
        }
```

```
    } catch as e {
        return {
            "url": url,
            "status": nil,
            "ok": false,
            "error": e.msg
        }
    }
}

func monitor(urls, timeout) {
    let results = []
    for url in urls {
        let result = check_url(url, timeout)
        results.push(result)
        let status = result["ok"] ? "UP" : "DOWN"
        print(url + ": " + status + (result["error"] != nil ? " (" + result["error"] +
    }
    return results
}

let parser = argparser.ArgumentParser("uptime-monitor")
parser.add_argument("urls", nargs="*", help="URLs to monitor")
parser.add_argument("--timeout", "-t", type="int", default=10, help="Timeout in seconds
let args = parser.parse_args()

if len(args.urls) == 0 {
    print("No URLs to monitor")
    sys.exit(1)
}

let results = monitor(args.urls, args.timeout)

let up_count = 0
for r in results {
    if r["ok"] {
        up_count = up_count + 1
    }
}
print("")
print("Summary: " + str(up_count) + "/" + str(len(results)) + " sites up")

// Save report
let report = json.dumps(results, {"indent": 2})
fs.write("uptime_report.json", report)
print("Report saved to uptime_report.json")
```

19.5 48.5 Running the Capstones

```
# Metadata archiver
cd book-v2/tests
export PYTHONPATH="$(../.venv/bin/python -c 'import sysconfig; print(sysconfig.get_path
export ZEN_NO_PY_BOOTSTRAP=1
ZEN=../../../target/release/zen

# Note: capstone scripts are examples; adapt paths as needed
# $ZEN run projects/capstone/archive_metadata.z https://example.com --output meta.json
```

19.6 48.6 Quick reference

```
# Web metadata archiver: requests + bs4 + json + fs
# Password checker: math + random + argparser + fs
# Log analyzer: fs + re + argparser
# Uptime monitor: requests + fs + argparser + json
```

19.7 48.7 Exercises

1. Extend the metadata archiver to also extract <h1> tags and meta keywords.
2. Add rate limiting to the uptime monitor (wait between checks).
3. Add email/Slack notifications to the uptime monitor when a site goes down.
4. Extend the password checker to check against a common-passwords list.
5. Add HTML report generation to the log analyzer.